



Resnet 논문정리 - 김지원

Introduction

? 더 나은 네트워크 학습하는 것이 레이어 층 더 쌓는 것만큼 쉬울까 ?

장애물: 기울기의 소실 및 대폭발 - 해결방법: 정규화, 중간 정규화

: 여러 레이어층도 back propagation 을 활용한 SGD 통해 수렴 가능

deeper networks 들이 수렴시작하면서 성능저하가 나타나기 시작함

: 네트워크가 깊어질수록 어느 시점 saturated(포화)되기 때문

— 예상과 달리—

1. 과적합에 의한 것이 아님
2. 층추가하면 오히려 training error 증가

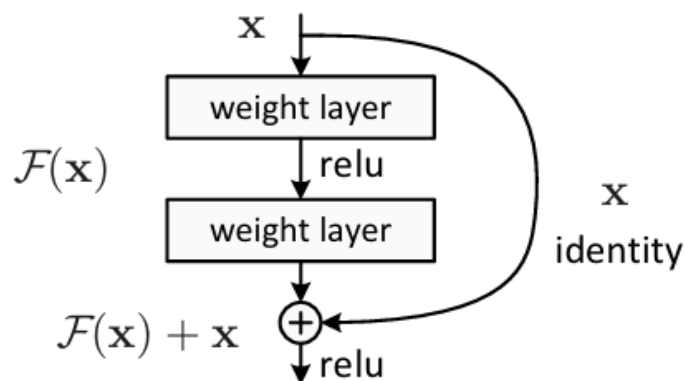


Figure 2. Residual learning: a building block.

추가된 layer = identity mapping

기존 layer = 학습된 shallow model의 layer 그대로 복사

이론적 결과 = 추가된 network의 training error 가 얇은 모델보다 작거나 같아야함

실제 결과 = 더 높아지기도 함

이번 논문에서는 성능저하 문제를 deep residual learning framework을 통해 해결함

-H(x) 전체를 바로 학습하는 대신 F(x)(입력에 무엇을 얼마나 더해야 하는지) 학습

$\tilde{H}(x) = x + F(x)$

그들의 가정: 원래 매핑 H(x)을 직접 최적화하는 것보다 잔차 매핑 $F(x) = H(x) - x$ 을 최적화하는 것이 더 쉽다고 가정

이유: 항등 매핑이 최적일 경우, 여러 비선형 층이 x를 그대로 학습하는 것보다 잔차 F(x)를 0으로 만드는 것이 더 쉽기 때문

Imagenet 을 통하여 다양한 실험을 함 → 성능저하 및 제안 방법 평가

Related work

1. Residual Representations

VLAD와 Fisher Vector : 원본 벡터 자체가 아니라 기준 벡터와의 차이인 residual vector를 인코딩하는 방법

- 이미지 검색과 분류에서 효과적인 shallow representation으로 사용됨
- 벡터 양자화에서도 원본 벡터보다 residual vector를 인코딩하는 것이 더 효과적인 것으로 나타남.
- PDE를 해결하는 Multigrid 방법 역시 문제를 여러 scale의 하위 문제로 나누고, 각 scale 사이의 residual solution을 구함. 이러한 residual 기반 방법은 residual을 고려하지 않는 기존 방법보다 더 빠르게 수렴한다.

→ 문제를 residual 형태로 재구성하면 optimization을 단순화하고 수렴 속도를 높일 수 있음

2. Shortcut Connections

ResNet 이전부터 MLP와 Inception 등의 모델에서 사용되어 옴

초기 MLP에서는 네트워크의 입력과 출력을 선형 layer로 직접 연결함

→ 이후에는 기울기 소실/폭발 문제를 해결하기 위해 중간 layer를 보조 classifier에 직접 연결하는 방법이 제안됨

→ 이때 Inception layer 역시 shortcut branch와 여러 개의 deeper branch로 구성

Highway Network와 ResNet의 차이

구분	Highway Network	ResNet
Shortcut 방식	Gate를 사용	Identity shortcut 사용
Parameter	Gate에 학습 가능한 parameter 존재	Shortcut에 parameter 없음
정보 전달	Gate가 정보 전달 여부를 결정	입력 정보가 항상 그대로 전달됨
Shortcut 차단	Gate가 닫히면 정보가 전달되지 않음	Shortcut이 항상 열려 있음
학습 대상	Residual function을 학습하지 않을 수도 있음	항상 residual function을 학습함
매우 깊은 모델	100층 이상에서 정확도 향상을 입증하지 못함	깊이가 크게 증가해도 정확도가 향상됨

→ ResNet은 parameter가 없는 identity shortcut을 통해 입력 정보를 항상 전달하면서 residual function을 추가로 학습

3. Deep Residual Learning

3.1 Residual Learning(Introduction 내용)

여러 nonlinear layer가 $H(x)$ 를 근사할 수 있다면, residual function $H(x)-x$ 도 근사할 수 있음 → $H(x)$ 를 직접 학습하는 대신 다음과 같이 residual을 학습

$$F(x)=H(x)-x, H(x)=F(x)+x$$

Deeper model의 추가 layer가 identity mapping을 수행하면 training error가 증가하지 않아야 하지만, 실제로는 nonlinear layer가 identity mapping을 학습하기 어려움.

Residual learning에서는 $F(x)$ 만 학습하므로, identity mapping이 최적일 때 weight를 0에 가깝게 만들어 $F(x)\approx 0$ 을 쉽게 구현할 수 있음

→ 새로운 함수 전체를 학습하는 것보다 identity mapping을 기준으로 작은 변화량인 residual을 학습하는 것이 더 쉽다.

3.2 Identity Mapping by Shortcuts

몇 개의 stacked layer마다 residual learning을 적용하며, 하나의 residual block은 다음과 같이 표현

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

- $\mathbf{x}$: residual block의 입력
- $\mathbf{y}$: residual block의 출력
- $\mathcal{F}(\mathbf{x}, \{W_i\})$: layer들이 학습하는 residual mapping
- $\mathbf{x}$: shortcut connection을 통해 그대로 전달되는 identity mapping

1. $\mathcal{F}(\mathbf{x})$ 와 $\mathbf{x}$ 는 element-wise addition으로 더해지고, 이후 ReLU를 적용

2. Shortcut connection은 추가 parameter와 계산량이 거의 없어, plain network와 ResNet을 동일한 depth, width, parameter 수 및 계산량으로 비교할 수 있다.

$\mathbf{x}$ 와 $\mathcal{F}(\mathbf{x})$ 의 차원이 다른 경우= linear projection W_s 를 사용해 차원을 맞춤

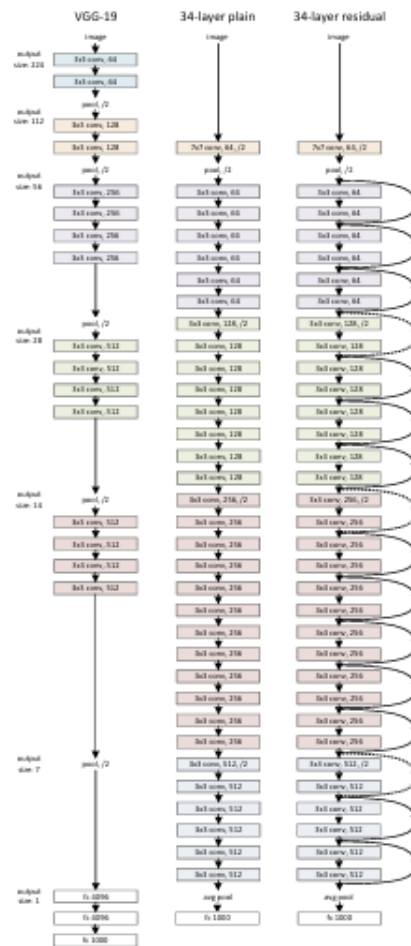
$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s \mathbf{x}.$$

Identity mapping만으로도 degradation problem을 해결할 수 있으므로, **W_s 는 차원을 맞춰야 할 때만 사용**

Residual function $\mathcal{F}$ 는 주로 2~3개의 layer로 구성되며, convolutional layer에도 동일하게 적용

→ ResNet은 parameter가 없는 shortcut을 통해 입력 $\mathbf{x}$ 를 그대로 전달하고, layer에서는 residual $\mathcal{F}(\mathbf{x})$ 만 학습한다.

3.3 Network Architectures



Plain Network

ImageNet 실험을 위해 VGGNet의 설계 방식을 참고하여 34-layer plain network를 구성

- 대부분의 convolutional layer에서 3×3 filter 사용
- Feature map 크기가 같으면 filter 수 유지
- Feature map 크기가 절반으로 줄면 filter 수를 두 배로 증가시켜 layer별 계산량 유지
- Stride 2인 convolution으로 downsampling
- 마지막에 global average pooling과 1,000-class fully connected layer 적용
- 계산량은 36억 FLOPs로, VGG-19의 약 18%

Residual Network

동일한 plain network에 shortcut connection을 추가하여 34-layer residual network를 구성

- 입출력 차원이 같음 → identity shortcut 사용
- 입출력 차원이 다름 → 두 가지 방법 사용

방법	차원을 맞추는 방식	추가 parameter
Option A	Identity mapping 후 부족한 차원에 0을 추가	없음
Option B	1×1 convolution을 이용한 projection shortcut	있음

Feature map 크기가 줄어드는 구간에서는 shortcut에도 stride 2를 적용

→ **Plain network와 residual network는 depth와 계산량이 같으며, 핵심적인 차이는 shortcut connection의 유무이다.**

3.4 Implementation

ImageNet의 데이터 전처리 및 증강 및 학습 과정은 기존 연구의 방식을 따랐으며, plain network와 residual network 모두 사전 학습 없이 처음부터 학습함. 각 convolutional layer 뒤, activation function 전에 BN을 적용했으며 dropout은 사용하지 않음.

4. Experiments

4.1 ImageNet classification

Plain Network

- 34-layer plain network는 18-layer보다 깊지만 training error와 validation error 가 모두 높게 나타남
- BN을 적용했음에도 이러한 현상이 발생했으므로, 이는 vanishing gradient가 아니라 깊은 plain network의 optimization이 어렵기 때문이라고 볼 수 있음

Residual Network

동일한 구조에 shortcut connection을 추가하자 34-layer ResNet이 18-layer ResNet 보다 높은 정확도를 보였으며 34-layer plain network보다 training error가 크게 감소

→ Residual learning은 깊은 네트워크의 optimization을 쉽게 하며, degradation problem을 해결

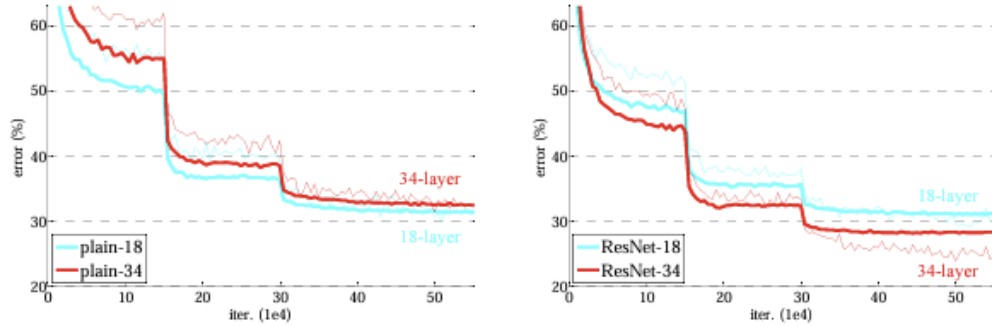


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

→ Plain network에서는 깊이가 증가하면 성능이 저하되지만, ResNet에서는 깊이가 증가할수록 training error와 validation error가 감소

Identity Shortcut vs. Projection Shortcut

방법	Shortcut 구성	특징
A	Identity shortcut + zero-padding	추가 parameter 없음
B	차원이 증가할 때만 projection 사용	필요한 구간에서만 차원을 맞춤
C	모든 shortcut에 projection 사용	추가 parameter와 계산량 발생

→ 세 방법 모두 plain network보다 좋은 성능을 보였으며, projection을 사용한 B와 C가 A보다 약간 높은 정확도를 보임. 그러나 성능 차이가 크지 않아, 경제성과 복잡도를 고려하면 parameter가 없는 identity shortcut만으로도 충분함.

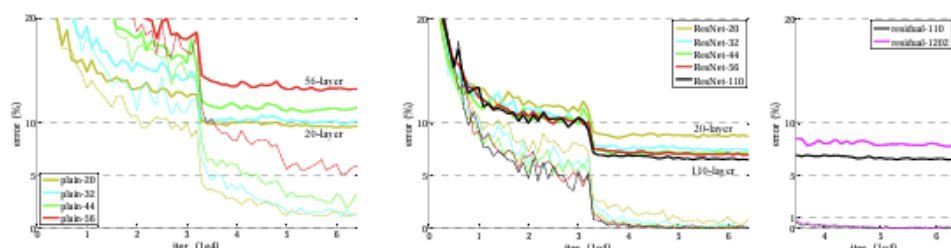
model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PRReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

→ Resnet 깊이 증가해도 degradation problem 없이 성능 향상되고 152 layer까지 안정적으로 학습 가능하다는 것 확인

4.2 CIFAR-10 and Analysis

CIFAR-10에서도 plain network는 깊어질수록 training error가 증가하여 degradation problem이 나타남. 반면 ResNet은 20-layer에서 110-layer까지 깊어질수록 성능이 향상됨



→ CIFAR-10에서도 plain network는 깊이가 증가할수록 성능이 저하되지만, ResNet은 깊어질수록 더 좋은 성능을 보임

1,202-layer ResNet도 optimization 자체는 가능했지만, 데이터셋 규모에 비해 모델이 지나치게 커 overfitting이 발생하면서 test error가 증가

→ ResNet은 매우 깊은 모델도 학습할 수 있지만, 모델의 깊이가 데이터셋 규모에 비해 지나치게 크면 overfitting이 발생할 수 있음.

Residual Response 분석

깊은 ResNet일수록 각 layer에서 학습된 residual function의 출력이 더 작게 나타남. 이는 깊은 block이 완전히 새로운 함수를 학습하기보다 identity mapping을 기준으로 작은 변화량을 학습한다는 것을 보여줌

4.3 Object Detection

ImageNet에서 학습한 ResNet을 PASCAL VOC와 COCO의 object detection 및 segmentation에 적용했을 때도 기존 모델보다 높은 성능을 보임



ResNet 논문 정리 - 손윤나

▼ 1. introduction

cnn은 low/mid/high 레벨의 feature와 classifier를 end-to-end 다층 구조로 통합함
→ 쌓인 레이어의 수(깊이)에 따라 feature의 수준이 풍부해짐.

네트워크 깊이가 매우 중요했고, 좋은 성능을 낸 모델들은 16~30층에 달하는 깊은 모델들이었음.

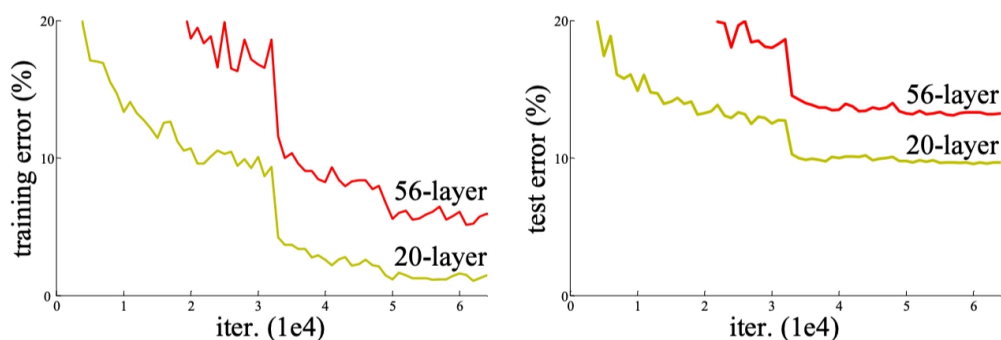
그러나 "레이어를 더 쌓기만 하면 더 좋은 네트워크를 학습시킬 수 있는가?"라는 질문이 제기됨. 이 질문에 충돌되는 것은 아래의 두가지와 같음.

- 기울기 소실/폭발 문제

학습의 수렴을 방해. → 그러나 이는 normalized initialization, batch normalization 등의 정규화 레이어로 상당 부분 해결 가능함. 따라서 수십 층의 네트워크더라도 sgd, 역전파로 수렴이 가능하게 됨.

- 성능 저하 문제

네트워크 깊이가 증가하면 정확도가 포화상태에 이르렀다가, 오히려 더 안좋아짐. 이는 과적합이 아니라, 이미 충분히 깊은 모델에 레이어를 더 추가하면 학습 오차 (training error)가 더 높아지는 현상임.



20번째 층보다 56번째 층의 training/test error가 더 높음.

→ 즉 모든 시스템이 똑같이 최적화하기는 어려움. 이론적으로는 깊은 모델이 얇은 모델보다 학습 오차가 높아질 이유가 없지만(construction 해가 존재하기 때문), 실험적으로는 최적화 알고리즘들이 더 나은 해를 찾아내지 못한다는 것이 드러남.

이 논문은 이러한 성능 저하 문제를 **deep residual learning 프레임워크**를 도입하여 해결함.

원하는 매핑 $H(x)$ 를 직접 근사하는 것이 아니라, residual mapping을 근사하도록 만들어서, $F(x) := H(x) - x$ 라는 다른 매핑을 학습하도록 함. 그리고 원래의 매핑은 $F(x) + x$ 로 recast됨.

$F(x) + x$ 는 shortcut connection(하나 이상의 레이어를 건너뛰는 연결)이 있는 feedforward 신경망으로 구현할 수 있음. 이 논문에서는 shortcut이 항등 매핑을 수행하고, 그 출력이 쌓인 레이어의 출력에 더해짐. 이는 추가적인 파라미터, 추가 연산 복잡도를 유발하지 않기 때문에, 여전히 sgd와 역전파로 학습이 가능함.

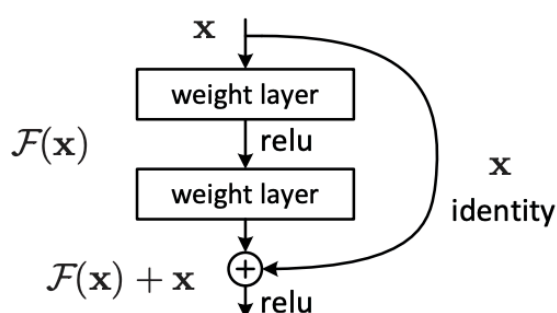


Figure 2. Residual learning: a building block.

실험을 통해 얻은 결론 (ImageNet, CIFAR-10 데이터셋)

1. 매우 깊은 residual network는 최적화하기 쉽지만, 단순히 레이어를 쌓기만 한 네트워크는 깊이가 늘어날수록 더 높은 학습 오차를 보임.
2. 깊은 residual network는 깊이가 늘어나도 정확도가 향상됨.

▼ 2. Relate Work

- residual representations

이미지 인식 분야에서 VLAD는 dictionary를 기준으로 한 잔차 벡터로 인코딩하는 표현 방식, Fisher Vector는 VLAD의 확률적 버전 → 이미지 검색/분류에서 얇은 방식임.

저수준의 vision과 computer graphics에서 PDE를 풀때 쓰이는 multigrid방법론은 시스템을 여러 스케일의 하위 문제로 재구성하는데, 각 하위 문제는 더 거친 (coarser) 스케일과 더 미세한(finer) 스케일 사이의 residual solution을 담당함. hierarchical basis preconditioning(계층적 기저 사전조건화)도 두 스케일 사이

의 잔차 벡터를 나타내는 변수에 의존함. → 잔차의 성질을 모르는 표준적인 solver 보다 훨씬 빠르게 수렴함.

즉, 좋은 재구성이나 사전조건화가 최적화를 단순화할 수 있다는 것이 시사됨.

- shortcut connections

MLP를 학습시킬 때 초기에는 입력에서 출력으로 연결되는 선형 레이어를 추가하거나, 기울기 소실/폭발 문제를 해결하기 위해 중간 레이어 몇개를 보조 분류기에 연결했음. 또다른 연구에서는 레이어의 response, gradient, 전파된 오차를 centering(중심화)하는 방법을 구현함.

highway networks는 gating function을 가진 shortcut connection을 제시함. 이 게이트들은 데이터에 종속적이고 파라미터를 가짐. 게이트가 있는 shortcut이 0에 가까워지면(단히면) highway network의 레이어는 비잔차 함수를 나타내게 됨. (사실상 해당 shortcut은 없는 것과 마찬가지가 됨)

반면 이 논문에서는 항상 잔차 함수를 학습하도록 했으며, 항등 shortcut은 절대 닫히지 않고 모든 정보가 그대로 전달되면서, 추가적인 잔차함수가 학습됨. 그리고 highway networks는 100층 초과와 같이 극도로 깊어지면 정확도 향상이 없음.

▼ 3. Deep Residual Learning

▼ 3.1. Residual Learning

몇개의 레이어가 근사해야 하는 매핑 $H(x)$, 그리고 x 가 이 레이어들 중 첫번째 레이어에 들어가는 입력이라고 하자.

여러 비선형 레이어가 복잡한 함수를 점근적으로 근사할 수 있다고 가정하면, 이는 레이어들이 $H(x) - x$ 를 점근적으로 근사할 수 있다고 가정하는 것과 동등함.

따라서 쌓인 레이어들이 $H(x)$ 를 근사하도록 하는 대신, 레이어들이 잔차함수 $F(x) := H(x) - x$ 를 명시적으로 근사하도록 함. 원래의 함수는 $F(x) + x$ 가 됨. 이 두 함수 점근적 근사가 가능한데, 학습의 난이도는 다름.

만약 추가된 레이어들을 항등 매핑으로 구성할 수 있다면, 더 깊은 모델은 얇은 대응 모델보다 학습 오차가 크지 않아야함. 성능 저하 문제는 solver들이 여러 비선형 레이어로 항등 매핑을 근사하는데 어려움을 겪고 있음을 시사함. residual learning을 사용하면, 항등 매핑이 최적인 경우 solver는 여러 비선형 레이어의 가중치를 0에 가깝게 만들어 항등 매핑에 접근하면 됨.

실제로 항등 매핑이 정확히 최적일 가능성은 낮지만, 이러한 재구성은 문제를 '사전조건화'할 수 있다는 점에서 도움이 됨. 최적 함수가 0 매핑보다 항등 매핑에 더 가깝다면, solver가 함수를 완전 새로 학습하는 것보다 항등 매핑을 기준으로 한 미세한 변

화를 찾는 것이 더 쉬울 것임. 실험에서도 학습된 잔차 함수들이 일반적으로 작은 응답을 가짐

(정리) 보통의 신경망들은 레이어들이 $H(x)$ 를 직접 배우도록 하지만, 대신 $F(x) = H(x) - x$ (목표에서 입력을 뺀 차이)를 배우도록 함. 그러면 최종 출력은 다시 입력을 더해준 $F(x) + x$ 가 됨. → $F(x)$ 가 0이 되도록 학습하면 됨. 가중치를 전부 0에 가깝게 만들면 $F(x) \approx 0$ 이 됨. 이는 입력을 그대로 복사하는 항등 함수를 만드는 것보다 훨씬 쉬움

▼ 3.2. Identity Mapping by Shortcuts

이 논문에서는 몇개씩 쌓인 레이어마다 residual learning을 적용함.

입력 x 가 weight layer → relu → weight layer를 거쳐서 $F(x)$ 가 되고, 다른 한쪽은 아무 변화 없이 그대로 x 가 빠져나감. 이 두 개가 끝에서 더해진 후, relu를 통과함.

$$y = \mathcal{F}(x, \{W_i\}) + x.$$

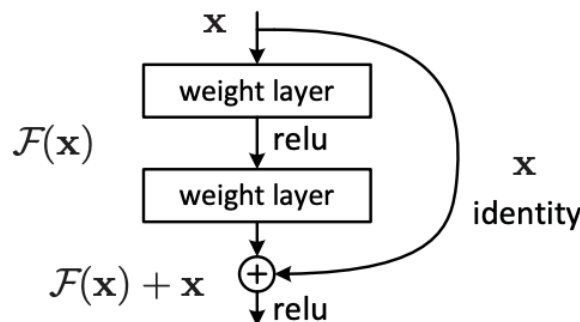


Figure 2. Residual learning: a building block.

Fig. 2처럼 레이어가 2개인 경우 $F = W_2\sigma(W_1x)$ 이고, σ 는 ReLU임. $F + x$ 연산은 shortcut connection과 원소별 덧셈으로 수행됨. 그리고 덧셈 이후 $\sigma(y)$ 를 적용함. 이러한 shortcut connection은 추가적인 파라미터, 연산 복잡도를 유발하지 않음. (plain과 residual 네트워크를 같은 선에서 비교할 수 있음)

위의 식에서 x 와 F 의 차원은 반드시 같아야 함. 그렇지 않다면 아래의 수식처럼 shortcut connection에서 선형 프로젝션 W_s 를 적용하여 차원을 맞춰야 함.

$$y = \mathcal{F}(x, \{W_i\}) + W_s x.$$

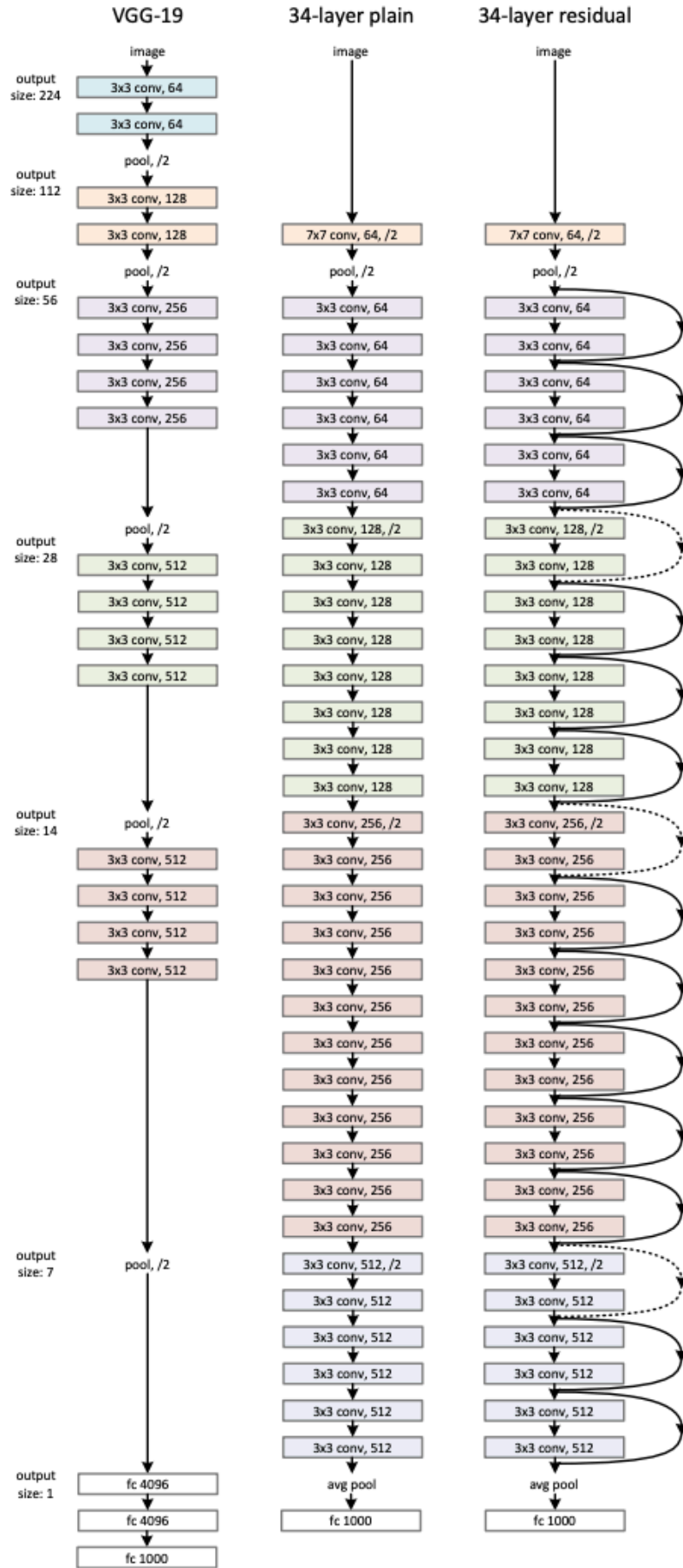
첫번째 식에서도 정방행렬 W_s 를 사용할 수 있지만, 항등 매핑만으로도 성능 저하 문제를 충분히 해결할 수 있고, 이를 안쓰는 것이 계산량 측면에서 경제적이기 때문에, W_s 는 차원을 맞출 때만 사용함.

이 실험에서 F 는 2 or 3개의 레이어로 구성되지만, 더 많은 레이어도 가능함. 다만 F 가 단 1개의 레이어만 가지면 사실상 선형 레이어($y = W_1x + x$)와 비슷해지기 때문에 이점이 없음.

이는 컨볼루션 레이어에도 적용 가능함. $F(x, \{W_1\})$ 는 여러 개의 컨볼루션 레이어를 나타낼 수 있고, 원소별 덧셈은 두 feature map에 대해 채널별로 수행됨.

▼ 3.3. Network Architectures

ImageNet 모델 비교



- VGG 네트워크
- plain network
 - 컨볼루션 레이어는 대부분 3*3 필터를 사용.
 - 동일한 출력 feature map 크기에 대해서는 레이어들이 동일한 개수의 필터를 가짐.
 - feature map 크기가 절반으로 줄어들면, 레이어당 시간 복잡도를 보존하기 위해 필터의 개수를 2배로 늘림.
 - 다운샘플링은 stride 2인 컨볼루션 레이어로 직접 수행함.
 - 네트워크는 global average pooling 레이어와 1000-way full connection + softmax 레이어로 끝남.

VGG 네트워크보다 필터 수가 적고 복잡도가 낮음. VGG-19(196억 FLOPs)의 18%.

- residual network
 - 위의 plain network를 기반으로 하여 3*3 컨볼루션 두 개마다 shortcut connection을 삽입함
 - 입력과 출력의 차원이 같을 때는 항등 shortcut을 그대로 사용할 수 있음.
 - 차원이 증가한다면 아래의 두 옵션 수행. shortcut이 서로 다른 크기의 두 feature map을 가로지를 때는 stride = 2
 - shortcut이 항등 매핑을 수행하되 늘어난 차원만큼 0을 추가로 채워넣는 방식(zero padding). 추가적인 파라미터가 필요 없음.
 - 프로젝션 shortcut을 사용하여 차원을 맞추는 방식.(1*1 컨볼루션으로 수행)

▼ 3.4. Implementation

이미지의 짧은 변이 [256, 480] 범위에서 무작위 샘플링 되어 크기가 조정됨.

이미지에서 224*224 크롭을 무작위 샘플링하고, 픽셀별 평균을 뺌.

색상 증강(color augmentation) 기법 활용

각 컨볼루션 직후, 활성화 함수 이전에 배치 정규화 적용.

가중치는 이전 연구 방식대로 초기화하고, 모든 plain/residual network를 처음부터 학습함.

미니배치 크기 256으로 SGD 사용.

학습률은 0.1에서 시작해서 오차가 정체될 때 1/10로 진행.

최대 60×10^4 반복까지 학습함

weight decay = 0.0001, momentum = 0.9

비교 연구를 위해 표준 10-crop 테스트를 채택.

fully - convolutional 채택.

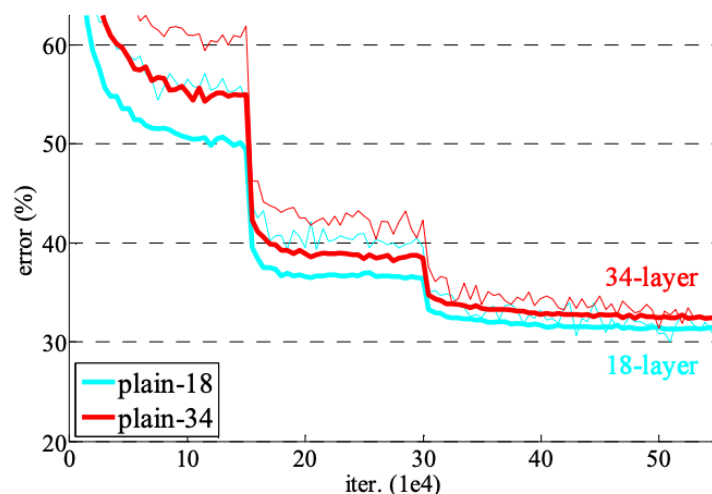
짧은 변이 {224, 256, 384, 480, 640}이 되도록 리사이즈한 이미지 여러 스케일에서 점수를 평균냄.

▼ 4. Experiments

▼ 4.1. ImageNet Classification

1000개 클래스로 구성된 ImageNet 2012 분류 데이터셋 활용함. 128만장 이미지로 모델 학습, 5만장의 validation 이미지로 평가함. 10만장의 테스트 이미지로 평가하여 오차율을 평가함.

- plain networks



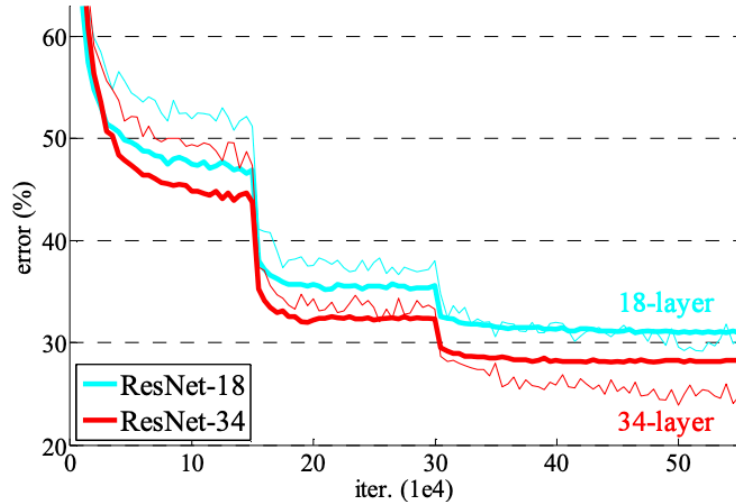
더 깊은 34-layer 네트워크가 더 얇은 18-layer 네트워크보다 검증 오차가 더 높음. 18-layer plain network의 해공간이 34-layer의 부분공간임에도 불구하고, 34-layer가 오차가 더 높음. (성능 저하 문제)

이 최적화 어려움이 기울기 소실 때문일 가능성은 낮음. 왜냐하면 plain network는 배치 정규화로 학습되어 순전파에서 분산이 0이 아니도록 보장되기 때문임. 역전파 기울기 역시 안정적임. → 순전파, 역전파에서 소실되지 않음.

실제로 34-layer network는 괜찮은 정확도를 달성하고 있는데, 이는 solver가 작동을 한다는 것을 의미함. 이 논문의 저자들은 깊은 plain network가 지수적

으로 낮은 수렴 속도를 가질 수 있고, 이것이 학습 오차 감소에 영향을 미친다고 추측함.

- residual networks



residual network의 베이스라인 구조는 plain과 동일한데 3*3 필터쌍마다 shortcut connection을 추가했음. 모든 shortcut에 항등 매핑을 사용했고, 차원이 늘어날 때 zero-padding을 사용했음.(plain 모델과 비교했을 때 추가 파라미터가 없음)

여기서 얻은 3가지 관찰은 다음과 같음.

- residual learning을 사용하면 34-layer이 18-layer보다 성능이 좋음 (2.85%p). 34-layer의 학습 오차가 상당히 낮고, 검증 데이터에도 잘 일반화됨. → 성능 저하 문제가 해결되고, 늘어난 layer로부터 정확도 향상을 얻어냄.
- plain 모델과 비교했을 때, 34-layer은 top-1 오차가 3.51%p 더 낮음. → 깊은 모델에서는 residual learning의 효과가 입증됨.
- 18-layer plain/residual network는 정확도가 비슷함(27.94%, 27.88%). 그러나 수렴은 residual이 더 빠름. → 네트워크가 그렇게 깊지는 않을 때(18-layer) SGD solver도 plain network에서 좋은 해를 찾을 수 있고, 이 경우에 ResNet은 더 빠르게 수렴함으로써 최적화를 도와줌.
- Identity vs. Projection Shortcuts

프로젝션 shortcut을 조사할 것인데, 아래의 3가지 옵션을 비교함.

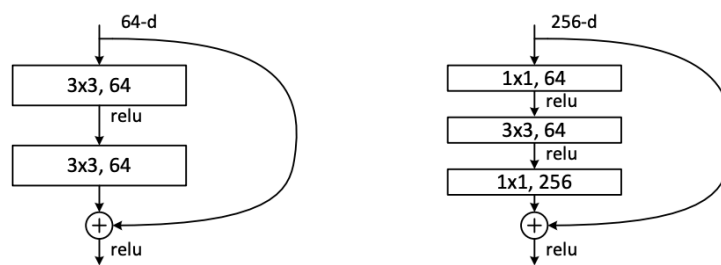
 - A: 차원 증가에 zero-padding shortcut 사용, 모든 shortcut은 파라미터가 없음.

- B: 차원 증가에 projection shortcut을 사용하고, 나머지는 identity shortcut
- C: 모든 shortcut이 projection

이 세 옵션 모두 plain 대응 모델보다 더 나옴. B가 A보다 조금 더 나운데, 이는 A에서 zero-padding된 차원 부분에서 residual learning이 일어나지 않기 때문임. C는 B보다 조금 더 나운데, 이는 projection shortcut이 추가한 파라미터 때문이라고 봄.

이후에는 메모리, 시간 복잡도, 모델 크기를 줄이기 위해 C 옵션을 활용하지 않음.

- Deeper Bottleneck Architectures



병목 설계 → 각 잔차 함수 F마다 2개 대신 3개의 레이어를 쌓은 스택을 사용함. 이 3개의 레이어는 각각 1×1 , 3×3 , 1×1 컨볼루션전이며, 1×1 레이어들은 차원을 줄였다가 다시 늘리는 역할을 하여 3×3 레이어가 더 작은 입출력 차원을 가진 병목이 되게 함.

파라미터가 없는 identity shortcut은 병목 구조에서 복잡도를 늘리지 않는다는 점에서 특히 중요함(identity shortcut은 병목 구조 설계에서 효율적인 모델). 위 Fig의 오른쪽의 Identity shortcut을 프로젝션으로 바꾸면, 시간 복잡도와 모델 크기가 두 배가 되기 때문임.

- 50-layer ResNet: 34-layer의 각 2-layer 블록을 3-layer 병목 블록으로 교체하여 구성하고, 차원 증가에는 B 옵션을 활용함 → 38억 FLOPs
- 101, 152-layer ResNet: 더 많은 3-layer 블록을 활용해서 구성했는데, 깊이가 늘었음에도 불구하고 152-layer ResNet의 113억이라는 FLOPs는 VGG-16/19의 153억, 196억 FLOPs보다 복잡도가 낮음.

이 3가지의 ResNet에서는 성능 저하 문제가 관찰되지 않았으며, 깊이가 늘어남에 따라 유의미한 정확도 향상도 보임.

- Comparisons with State-of-the-art Methods

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PreLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PreLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

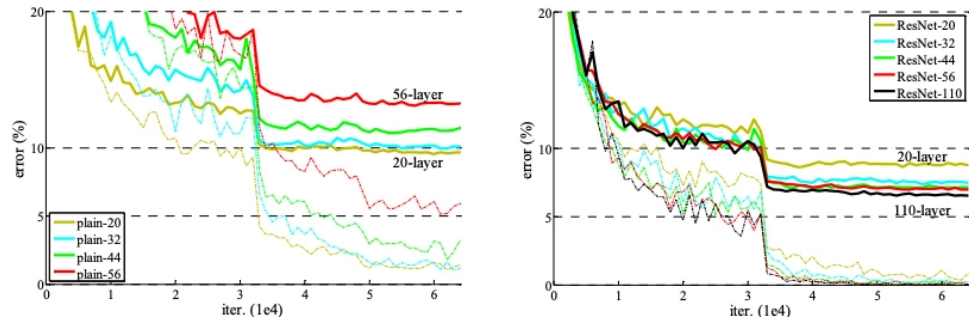
- ResNet-152 단일 모델 기준 top-5 검증 오차 4.49% 기록함. 이는 이전의 다른 앙상블 결과들을 능가함. 서로 다른 깊이의 모델들을 결합해 만든 ResNet 앙상블 모델은 top-5 오차 3.57%를 달성하며 ILSVRC 2015 1위 차지.

▼ 4.2. CIFAR-10 and Analysis

- 실험 환경
 - CIFAR-10 데이터셋(학습 5만장, 테스트 1만장, 10개 클래스)에서 추가 연구를 진행했음. 이 연구에서는 극도로 깊은 네트워크의 행동을 살펴보고자 함.
 - 네트워크 입력은 32*32 이미지, 픽셀별 평균을 뺌. 첫 레이어는 3*3 컨볼루션이고, 그다음 {32, 16, 8} 크기의 feature map에 대해서 각각 2n개씩, 총 6n개의 3*3 컨볼루션 레이어 스택을 사용함. 필터 개수는 각각 {16, 32, 64}. 서브 샘플링은 stride = 2로 수행, 네트워크는 global average pooling, 10-way fully connected, softmax 출력. → 총 6n+2개의 가중치 레이어

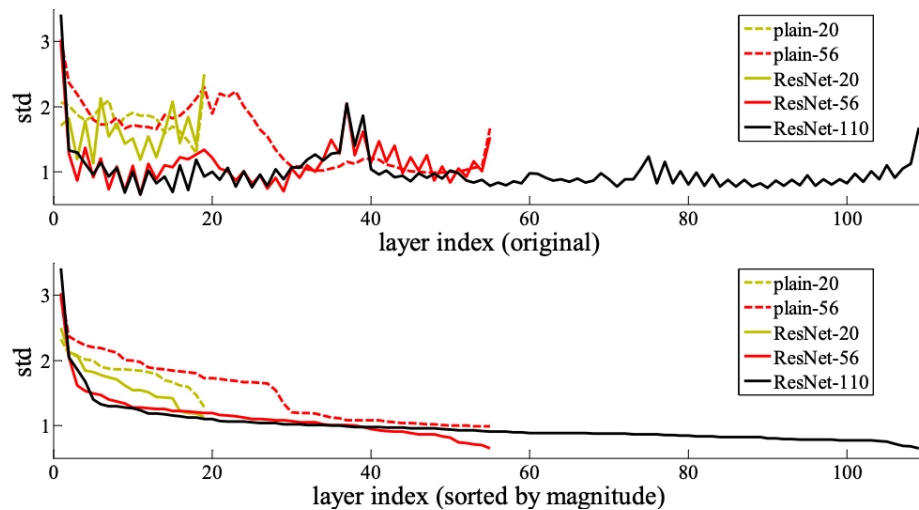
output map size	32×32	16×16	8×8
# layers	1+2n	2n	2n
# filters	16	32	64

- shortcut connection은 3*3 레이어 쌍마다 연결하여 사용.
- 이 데이터셋에서는 identity shortcut(A 옵션)을 사용. (residual과 plain 모델은 동일한 깊이, 너비, 파라미터 수를 가짐)
- 실험
 - n = {3, 5, 7, 9}, 즉 20, 32, 44, 56층 네트워크를 구성하였음.



- 왼쪽 사진은 plain 네트워크: 깊이가 늘어날수록 학습에 어려움을 겪으며, 깊어질수록 높은 학습오차를 보임. → 최적화 어려움
- 가운데 사진은 ResNet: 깊이가 늘어날수록 정확도 향상을 보여줌. $n = 18$ 로 하여 ResNet-110도 탐구했는데, 초기 학습률 0.1이 수렴하기에 좀 커서, 학습오차가 80% 아래로 떨어질 때까지 0.01로 학습하다가, 그 이후는 0.1로 하여 학습시킴. → 다른 깊고 얇은 네트워크보다 파라미터가 적으면서도 오차 6.43% 달성함.

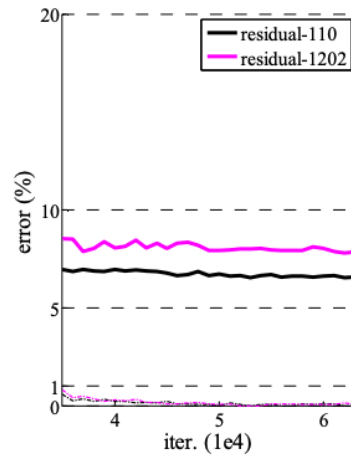
• Analysis of Layer Responses



- 레이어 응답의 표준편차를 나타낸 Figure. 응답은 각 3×3 레이어의 출력으로, 배치 정규화를 거치고 ReLU 덧셈 이전의 값임.
- 일반적으로 ResNet은 plain 모델보다 더 작은 응답을 가짐. 잔차 함수가 비잔차 함수보다 0에 더 가깝다는 3.1절을 뒷받침함.
- 더 깊은 ResNet일수록 응답의 크기가 더 작음. 레이어 수가 더 많을수록 개별 레이어가 신호를 덜 수정하는 경향이 있음.

• Exploring Over 1000 layers

n = 200으로 하여 1202 layer의 네트워크를 만들었고, 0.1% 미만의 학습 오차를 달성함. (테스트 오차 7.93%). 최적화도 잘 됨.



그러나 1202-layer 모델 테스트 결과는 110-layer보다 좋지 않음. 이 논문의 저자들은 이것이 과적합 때문이라고 주장함. 1202-layer는 실험에 사용한 작은 데이터셋에 비해 불필요하게 크기 때문임. 따라서 maxout이나 dropout같은 정규화를 적용해야 해당 데이터셋에서 좋은 결과를 얻을 수 있음.

이 논문에서는 maxout/dropout을 사용하지 않고, 단순히 깊고 얇은 구조로만 설계하여 정규화 효과를 얻음.(최적화의 어려움이라는 논문의 핵심에서 벗어나지 않기 위하여)

▼ 4.3. Object Detection on PASCAL and MS COCO

이러한 방법은 다른 태스크에서도 좋은 일반화 성능을 보임.

Faster R-CNN을 탐지 방법으로 채택하고, VGG-16, ResNet-101을 사용했는데, 그 결과를 보면 ResNet-101이 성능이 더 개선됨.

어려운 COCO 데이터셋에서 6.0%의 향상을 했는데, 이는 28%의 상대적 성능 향상임.



Resnet 논문정리 - 채

기존에는 residual learning을 썼는데 deep neural net은 더 train하기 어려웠다.

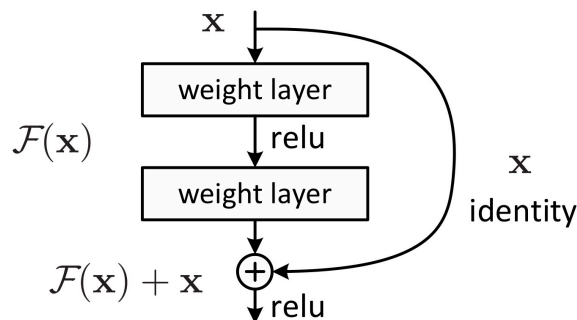
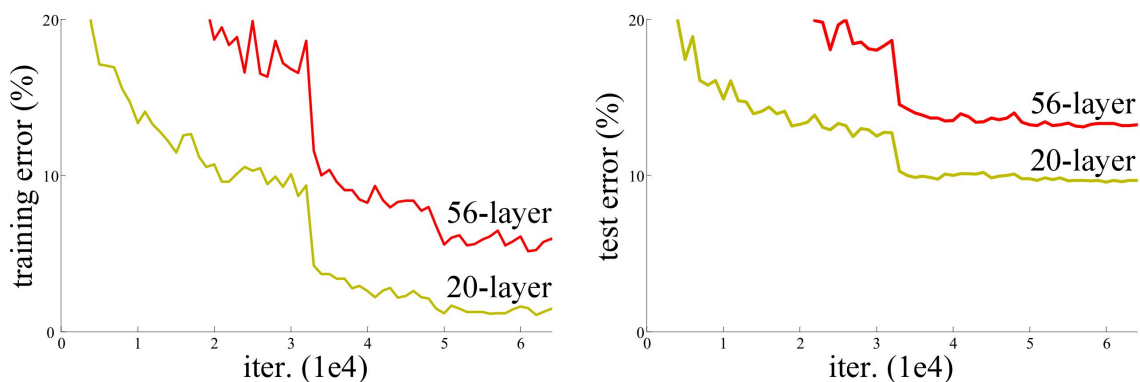


Figure 2. Residual learning: a building block.

Is learning better networks as easy as stacking more layers?
vanish, explode gradient가 문제였는데
normalized initialization으로 해결.



아니다. 단순히 stacking 만 해서는 성능이 더 좋아지지 못함.

이 논문에서는 Degradation problem을 deep residual learning으로 해결한다.

기존의 방식은 $H(x)$ 를 desired underlying mapping으로 사용했는데 Deep Residual Learning에서는 $F(x)$ 사용

$$F(x) = H(x) - x$$

F 가 최적화하기가 더 쉬움.

$H(x) = F(x) + x$ 는 Feedforward nn($F(x)$) with shortcut connections(x)으로 이해가능.

Image net 실험에서의 발견

1. Deep residual net은 easy to optimize
2. In plain network, when the depth is increased, training error increased.
3. Deep residual net 은 deep 한 구조에서도 high accuracy

이러한 결과는 CIFAR-10 set에서도 발견됨.

즉 vision & non-vision problem에 모두 적용가능.

Related Work.

Residual Representation.

Reformulation, Preconditioning이 최적화를 더 쉽게 할 수 있다!!

이미지 인식, 편미분방정식, Multigrid method 등에 사용가능.

Shortcut connection.

Inception layer는 shortcut branch와 deeper branch로 구성.

Highway network는 shortcut connections with gating function.

Deep Residual Learning

3.1 Residual Learning

$F(x) = H(x) - x$ (목표에서 입력을 뺀 차이)를 배우도록 함.

이게 학습의 난이도 측면에서 훨씬 쉽다.

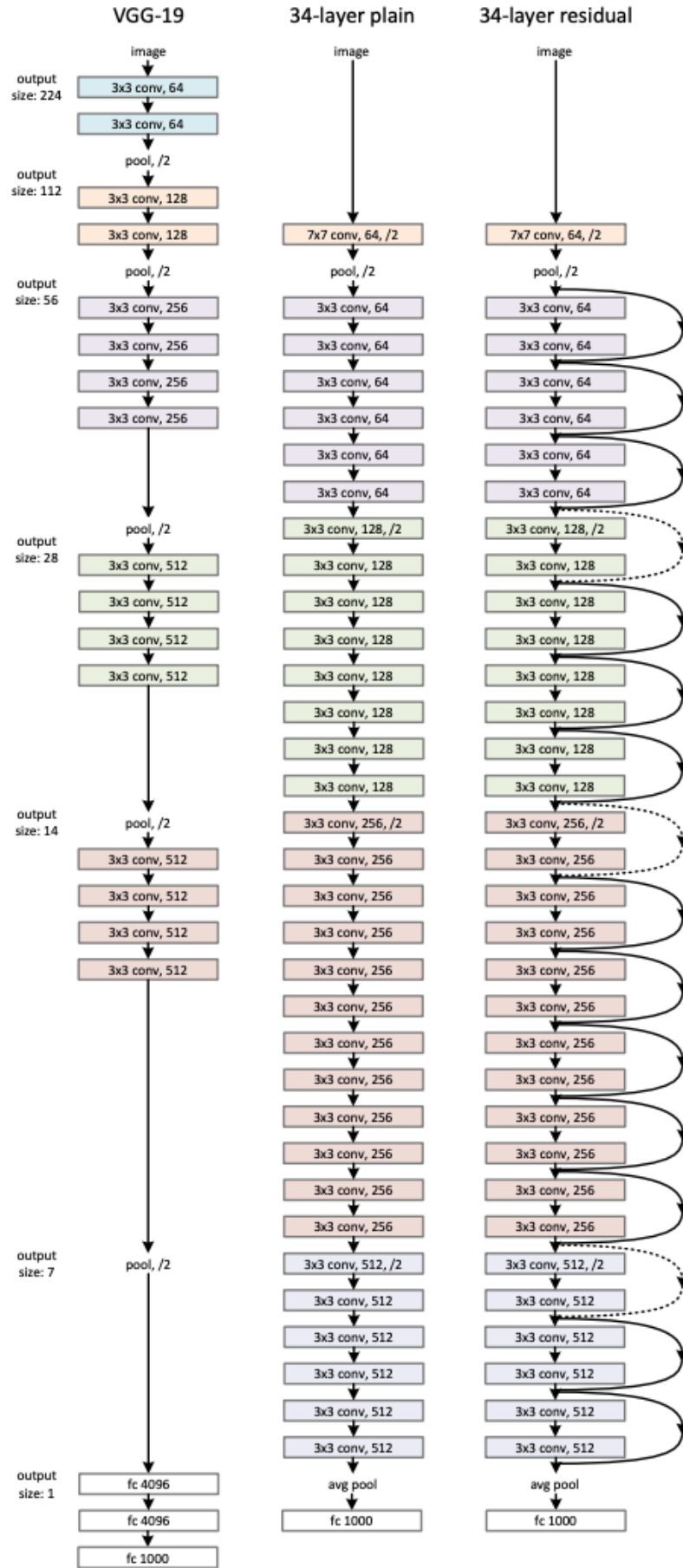
3.2 Identity mapping by shortcuts

$$y = \mathcal{F}(x, \{W_i\}) + x.$$

앞에 F 함수는 residual mapping, 뒤에 x 가 shortcut.

identity mapping은 degradation problem을 설명하는데 충분하고 효과적이다.

3.3 Network Architectures



Plain network는 vgg 모델을 많이 따옴.

3*3 filter → same output feature map size, same number of fillers, preserve the time complexity per layer를 하기 위해서.

Residual network는 plain network에 shortcut connection이 추가된 형태.

3.4 Implementation

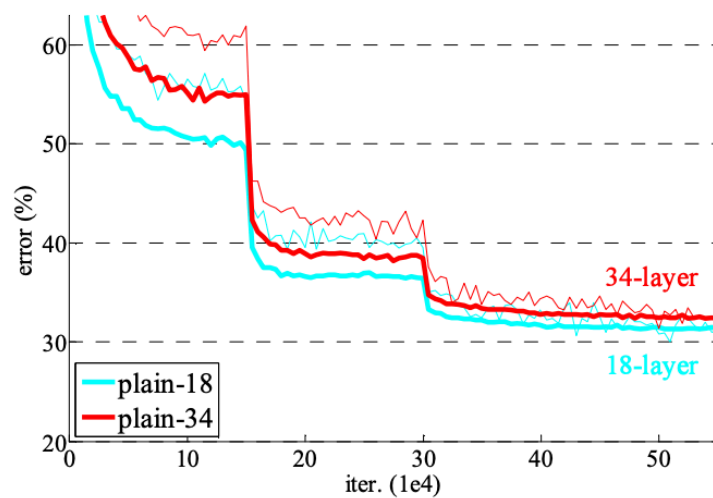
4. Experiments

4.1 Imagenet Classification

1.28M train image data

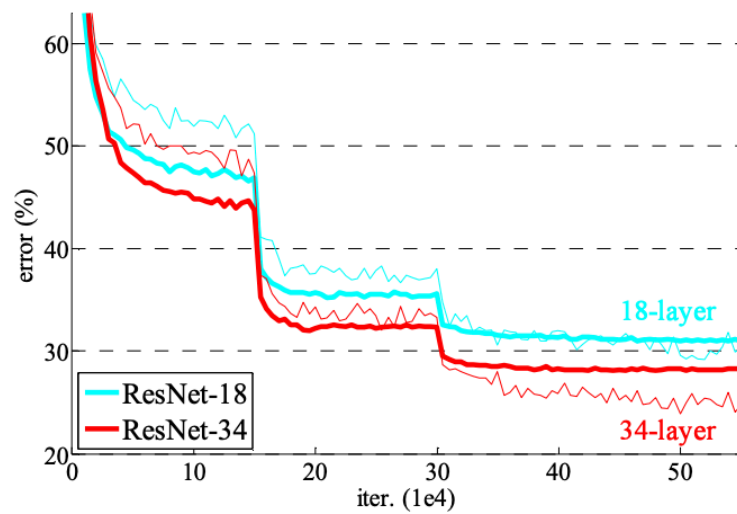
50k validation image data

100k test image data



plain model의 경우 층을 많이 쌓으니 오히려 안좋은 결과.

vanishing gradient때문은 아닌게 batch normalization으로 해결함. 단지 18층보다 수렴 속도가 느린 것.



residual learning을 진행한 결과, 34층이 더 좋은 성능.

층이 깊어지면 발생하는 degradation문제를 residual learning으로 잘 해결함. 추가 파라미터도 없음.

18layer plain vs 18 layer resnet.

resnet이 좀 더 빠르게 수렴한다.

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Error rates (%. **10-crop** testing) on ImageNet valid

A) parameter free인 zero padding shortcut으로 dimension increased.

B) projection shortcut으로 차원 증가. shortcut은 identity identity라서 모델 복잡도 증가시키지 않음.

C) 모든 shortcut은 projection.

plain보다 A~C 모든 방식이 더 좋음. $B \geq C > A > \text{plain}$

Deeper bottleneck architectures

$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ conv layer일때

1×1 레이어들은 차원을 줄였다가 다시 늘리는 역할을 하고
 3×3 레이어에서 병목이 발생.

4.2 CIFAR-10 and Analysis

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72 $\pm$ 0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61 $\pm$ 0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean $\pm$ std)” as in [43].

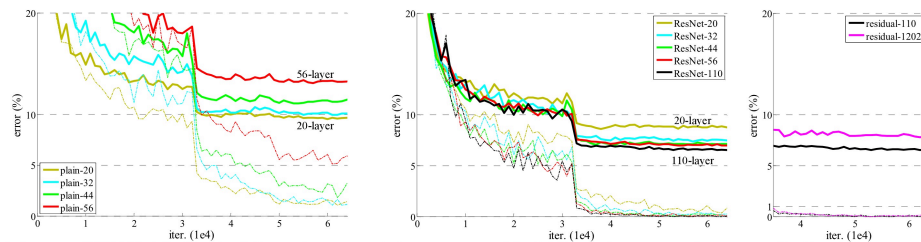


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

위와 동일한 결과. plain 인 왼쪽 그래프는 층을 쌓을수록 성능이 저하. resnet인 가운데 그래프는 층을 쌓을수록 성능 증가. 그러나 오른쪽 그래프를 보면 110층 vs 1202층 resnet을

비교했는데 train error는 비슷하지만 test error는 110층이 더 나음. resnet도 층을 무한히 쌓을 수 있는 것은 아님.

Strong regularization을 모델에 적용하면 성능이 더 개선될 여지가 있다. 그러나 이 논문에서는 단순한 정규화(no maxout, no dropout 등)만 사용.

4.3 Object Detection on PASCAL and MS COCO

Faster R-CNN system 사용.

Imagenet 분류 모델로 initialize하고 물체 탐지 목적에 맞게 fine tuning.

No hidden fully connected layers.

MS COCO Dataset

80종류의 물체 카테고리.

Box refinement, Global context, Multi-scale testing으로 성능향상.

1. Deep Residual Learning for Image Recognition (ResNet)

고려대 통계대학원 25 / KUBIG 23기 조호평

저자	Kaiming He, et al.
핵심 키워드	ResNet, Skip Connection, Residual Block

1. 논문 3줄 요약

2. Introduction : 인공지능망이 깊어지면 무조건 좋을까?

3. 논문 심층 분석 1 - Abstract

4. 논문 심층 분석 2 - Introduction(!!!매우 중요!!!)

5. 논문 심층 분석 3 - Deep Residual Learning

5.1. Residual Learning은 왜 $H(x)$ 대신 $F(x)$ 를 배우는가?

5.2. Identity Mapping by Shortcuts

5.3. <TIP> : F 가 1개의 층만 있으면?

5.4. Network Architecture

5.5. 더 직관적인 예시

5.6. <Tip & Question>

수식으로 보기 (선형 결합)

5.7. Implementation

6. 논문 심층 분석 4 - Experiments

6.1. Table 1 상세 분석 : 아키텍처 구성 원리

6.1.1 표기법 해석 (ResNet-34를 중심으로)

6.1.2. 구조적 차이점 (ResNet-50 예시)

6.1.3. 전체 층수(Depth) 계산

6.2. 실험 결과

6.2.1. 퇴보 문제의 확인 (Plain Networks)

6.2.2. 잔차 학습의 효과 (Residual Networks)

6.2.3. 최신 모델과의 비교 (SOTA 달성)

6.3. 중요 개념 소개 - Identity vs. Projection & Bottleneck

6.3.1. 추가 실험 : Shortcut 옵션 비교

6.3.2. 심층 모델을 위한 Bottleneck 구조

6.3.3. 병목 구조의 효율성 원리 (Why Bottleneck?)

6.3.4. Identity Shortcut의 필수성

6.3.5. <Tip> 더 쉬운 비유로 이해해보자!

6.4. 최종 성능 및 SOTA 비교 (Comparisons with SOTA)

6.5. 추가 분석 : CIFAR-10 및 층 반응(Layer Response)

7. 결론 및 제언

7.1. 핵심 요약 (Summary)

7.2. 논문의 주요 기여점 (Main Contributions)

7.3. 통계학적/분석적 고찰 (Discussion)

8. 부록 : 파이토치(PyTorch) 구현 예시

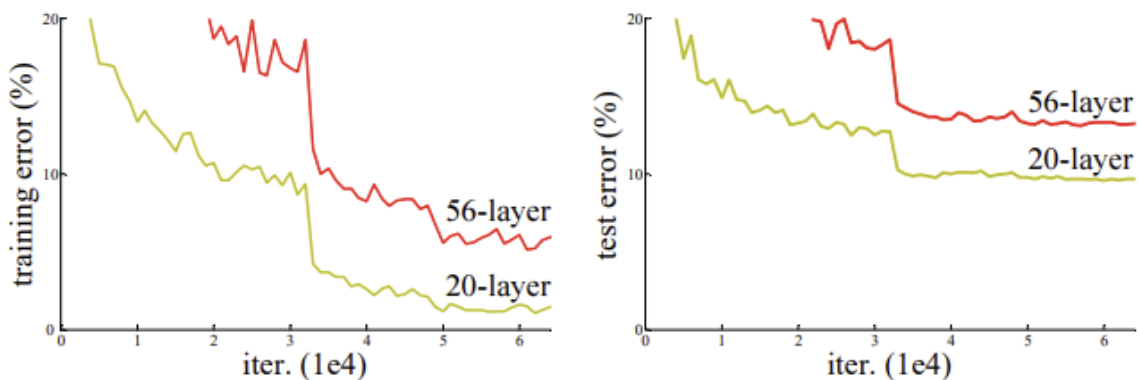
1. 논문 3줄 요약



- **문제(Problem)** : 신경망이 깊어질수록 성능이 좋아질 것 같지만, 실제로는 학습이 안 되고 에러가 커지는 Degradation Problem이 발생함.
- **해결(Solution)** : 입력을 출력에 바로 더해주는 **Skip Connection**을 도입해, 모델이 **Residual**만 학습하게 만듦으로써 깊은 망도 쉽게 최적화할 수 있게 됨.
- **결과(Result)** : 기존보다 훨씬 깊은 **152층 모델**을 성공적으로 학습시켜, ILSVRC 2015 대회에서 우승하고 사람보다 뛰어난 인식률을 처음으로 기록하게 됨.

2. Introduction : 인공신경망이 깊어지면 무조건 좋을까?

- **기존의 상식** : 우리는 처음 신경망을 배웠을 때 직관적으로 층(Depth)이 깊어질수록 더 복잡한 특징(Feature)을 추출할 수 있기에 성능이 좋아져야 한다고 생각했다.
- **현실 (Degradation Problem)** : 막상 층을 20층에서 56층으로 늘려보니, 오히려 학습 에러(Training Error)와 테스트 에러(Test Error)가 모두 높아지는 Underfitting 현상이 발생했다. 즉, 깊은 모델은 **학습(Optimization) 자체가 어렵다**는 것이 문제였다.



깊은 망(56-layer)이 얇은 망(20-layer)보다 에러가 더 높음

3. 논문 심층 분석 1 - Abstract

- **Unreferenced vs Referenced (참조 여부)**
 - **기존(Unreferenced)** : 입력 x 를 무시하고 출력 $H(x)$ 를 0부터 새로 만들려고 함. (어려움)
 - **제안(Referenced)** : 입력 x 를 참조해서(기준으로 삼아서), 차이 $F(x)$ 만 학습함. (쉬움)
- **8x deeper but lower complexity (더 깊은데 더 가볍다)**
 - 상식적으로 층이 8배 깊으면 계산도 8배 많아야 한다.
 - 하지만 ResNet은 병목 구조(Bottleneck Architecture)라는 테크닉을 써서 연산량을 획기적으로 줄였다. 그래서 깊지만 가볍다는 장점이 있다.
- **Empirical Evidence (경험적 증거)**
 - 수학적으로 완벽하게 증명했다기보다, 실험해보니 실제로 되더라는 것을 강조했다.

4. 논문 심층 분석 2 - Introduction(!!!매우 중요!!!)

4.1. 배경 : 깊이(Depth)의 승리

- 최근 ImageNet 대회 상위권 모델들(VGG 등)은 모두 16~30층의 깊은 구조를 가짐. 층이 깊을수록 특징(Feature)을 더 풍부하게 추출할 수 있음이 증명됨.

4.2. 과거의 문제점: 기울기 소실 / 폭발

- **질문** : 그럼 무조건 깊게만 쌓으면 장땡인가?
- **과거의 난관** : 층이 깊어지면 역전파 과정에서 미분값(Gradient)이 사라지거나 폭발해서 학습 자체가 안 되는 **Vanishing/Exploding Gradient** 문제가 있었음.
- **해결 상태** : 이 문제는 초기화 기법(Initialization)과 **Batch Normalization** 등의 등장으로 어느 정도 해결되어, 일단 학습(수렴) 시작은 가능해진 상태임.

4.3. Solution

- **퇴보 문제 (Degradation Problem)** : 층이 깊어질수록 정확도가 오히려 떨어지는 현상이 발견됐다.
- **중요한 발견** : 이것은 **과적합(Overfitting)**이 아니다. (학습 에러도 같이 높아지기 때문에)
- **논리적 모순** : 얇은 모델에 *Identity* 층만 추가해도 깊은 모델이 되므로, 이론상 깊은 모델의 에러가 더 높을 수 없다. 즉, 모델의 용량(Capacity) 문제가 아니라 **학습 (Optimization)** 난이도의 문제인 것이다.

4.4 핵심 아이디어

- **기존 방식**: $H(x)$ 를 직접 학습하려고 함. (어려움)
- **제안 방식 (Residual Learning)** : $H(x) - x$, 즉 잔차 $F(x)$ 를 학습함. 최종 출력은 $F(x) + x$.
- **왜 쉬운가?**: 만약 최적의 해가 *Identity*라면, 기존 방식은 복잡한 연산으로 x 를 만들어 내야 하지만, 잔차를 이용한 방식은 **가중치를 0으로만 보내면**($F(x) \rightarrow 0$) 되기 때문에 훨씬 쉽다.

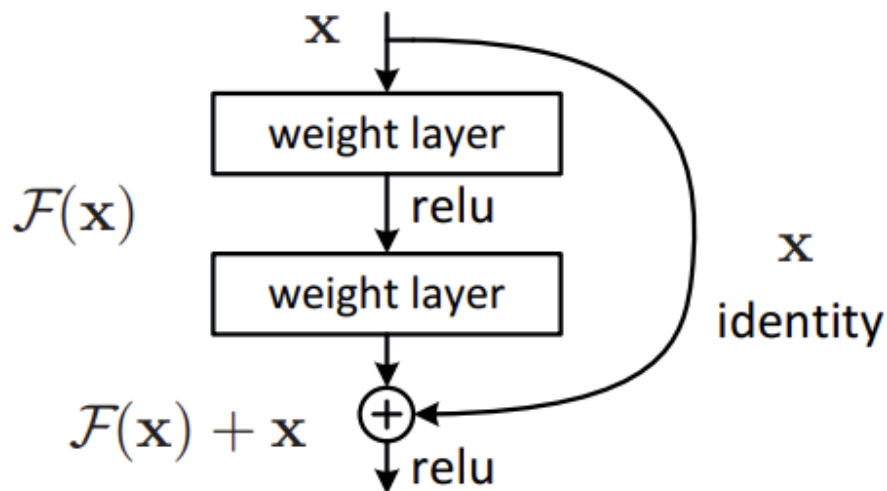


Figure 2. Residual learning: a building block.

ResNet의 기본 단위인 Residual Block. 오른쪽으로 흰 화살표가 바로 Skip Connection이다.

다음 밑의 간단한 공식을 보자.

$$H(x) = F(x) + x$$

기존의 신경망은 $H(x)$ 를 직접 학습하려고 했다. 하지만 ResNet은 $F(x) = H(x) - x$ (잔차, Residual)를 학습하는 것으로 관점을 바꾸었다. 즉, 입력 x 와 정답 $H(x)$ 의 차이 $F(x)$ 만 학습하자는 아이디어다.

이유 : 만약 최적의 해가 '아무것도 안 하는 것(Identity Mapping, x)'이라면, 기존 방식은 복잡한 비선형 층들을 학습시켜 x 를 만들어내야 해서 비효율적이었다. 하지만 ResNet 구조에서는 **학습해야 할 $F(x)$ 를 0으로 만들어버리면** 되기 때문에 훨씬 쉽다. (Weight를 0으로 수렴시키는 게 더 쉬움)

밑에서 더 쉬운 설명으로 다시 한 번 이해하도록 노력해보자.

4.5. 왜 이름이 잔차(Residual)인가? (통계학적 접근)

우리가 잘 아는 회귀분석을 떠올려 보자.

$$y = \hat{y} + \epsilon \quad (\text{관측값} = \text{예측값} + \text{잔차})$$

여기서 잔차(Residual)는 모델이 아직 설명하지 못한 나머지 오차를 뜻한다.

ResNet에서도 똑같다.

- x (**Input**) : 이전 층에서 이미 학습해온 정보 (내가 이미 가지고 있는 지식).
- $H(x)$ (**Target**) : 이번 층에서 최종적으로 만들어내야 할 완벽한 정답.
- $F(x)$ (**Residual**): x 만으로는 부족해서, 추가로 더 학습해야 하는 나머지 차이.

즉, " $H(x)$ 전체를 땀으로 배우려니 너무 힘들니까, 기존 지식(x)은 그대로 두고 그 차이 $F(x)$ 만큼만 살짝 더 배우자!" 라는 뜻에서 잔차 학습(Residual Learning)이라고 부르게 된 것이다.

4.6. 왜 이게 더 쉬운가? (철학적 이유)

이걸 직관적인 예시로 설명해 보자.

상황 : 내가 기말고사 시험 공부를 한다고 치자.

- **기존 방식 (Plain Network):**

- 내 실력이 80점(x)인데, A+를 위해 목표를 95점 $H(x)$ 으로 잡았다.
- 기존 방식은 80점인 나를 싹 지우고, 처음부터 95점짜리 지식을 통째로 다시 만들려고 한다. (0부터 95 만들기) -> 상식적으로 당연히 어렵고 비효율적이다.

- **ResNet 방식 (Residual Learning):**

- 내 실력 80점(x)은 어차피 기존 실력이 있으니깐 변하지 않으므로 그대로 보존한다 (Skip Connection).
- 그리고 부족한 15점 $F(x)$ 만 추가로 공부해서(어려운 파트를 더 집중적으로 공부한다든지) 더한다. (80 + 15 만들기) -> **훨씬 쉽다!**

4.7. 수학적 이유 (Identity Mapping)

딥러닝에서는 층이 깊어지면 가끔은 아무것도 안 하고 그냥 통과시키는 것(Identity Mapping, x)이 최선일 때가 있다.

- **기존 방식:** $H(x) = x$ 를 만들려면, 가중치(Weight)를 아주 정교하게 조절해서 입력을 그대로 복사하는 함수를 만들어야 한다. (수학적, 컴퓨팅적으로 어려움)
- **ResNet:** $H(x) = F(x) + x$ 구조니까, 그냥 $F(x)$ (**학습할 가중치**)를 0으로 만들어버리면 된다. ($0 + x = x$, 여기서 $F(x) = 0$ 이 돼 학습하지 않았다).
- 가중치를 0으로 수렴시키는 것이 특정 값을 맞추는 것보다 학습 난이도가 훨씬 낮다.

4.8. 요약 : 이 논문이 증명한 것

- **최적화 용이성:** 일반 네트워크(Plain Net)와 달리, ResNet은 층이 깊어져도 학습 어려움이 낮아지며 최적화가 쉽다.

- **범용성(Generalization)** : 단순히 분류(Classification)만 잘하는 것이 아니라, 탐지(Detection), 세그멘테이션(Segmentation) 등 모든 대회에서 1위를 휩쓸 만큼 특징 추출 능력이 뛰어나다.
- **효율성**: 152층으로 깊이는 VGG보다 8배 깊지만, 연산 복잡도는 오히려 더 낮다.

5. 논문 심층 분석 3 - Deep Residual Learning

5.1. Residual Learning은 왜 $H(x)$ 대신 $F(x)$ 를 배우는가?

- **가설의 전환** : 복잡한 함수 $H(x)$ 를 통째로 배우는 것보다, 입력과의 차이인 잔차 $F(x)$ 를 배우는 것이 학습 난이도가 낮다.
- **Identity Mapping의 용이성** : 모델이 깊어질수록 입력값을 그대로 보존하는 것 (Identity)이 중요한데, 기존 방식은 이를 배우기 어렵다. 하지만 ResNet 구조에서는 **가중치(Weight)를 0으로만 만들면** 수학적으로 자동적으로 Identity(x)가 되므로 최적화가 훨씬 쉽다.
- **사전 조건화 (Preconditioning)** : 설령 정답이 완전한 Identity가 아니더라도, Identity에 가까운 상태에서 시작하여 조금씩 수정(Perturbation)해 나가는 것이 맨땅에서 학습하는 것보다 훨씬 효율적이다.

5.2. Identity Mapping by Shortcuts

- **기본 수식**
 - $y = F(x) + x$
 - x : 입력, $F(x)$: 잔차 함수(학습 대상), y : 최종 출력
 - 구조 : Convolution $\rightarrow$ ReLU $\rightarrow$ Convolution $\rightarrow$ **Addition (x 더하기)** $\rightarrow$ ReLU
- **Shortcut의 장점 (Efficiency)**
 - **파라미터 0**: 입력 x 를 그대로 더하기만 하므로, 학습해야 할 파라미터가 전혀 늘어나지 않는다.
 - **연산량 0** : 덧셈 연산은 행렬 곱셈에 비해 무시할 수 있을 만큼 가볍다.
 - **결론** : 기존 네트워크와 동일한 파라미터 수로 훨씬 깊은 망을 만들 수 있다.
- **차원(Dimension)이 다를 때의 처리**
 - 기본적으로 x 와 $F(x)$ 의 사이즈(Channel 수)가 같아야 더할 수 있다.
 - 만약 채널 수가 다르면(예: $64 \rightarrow 128$)?

◦ W_s (**Linear Projection**): 1×1 Convolution 등을 사용하여 x 의 차원을 키워준 뒤 더한다. ($y = F(x) + W_s x$)

5.3. <TIP> : F 가 1개의 층만 있으면?

- F 는 적어도 2개 이상의 층(Layer)이어야 한다는 말이 본문에 있는데, 만약 층이 1개라면 어떻게 될까?

$y = W_1 x + x$ 식을 잘 보면 이건 그냥 선형 결합(Linear Layer)이랑 똑같아져 버린다. 비선형성(Non-linearity)이 사라져서 깊게 쌓는 의미가 없어지는 것이다. 그래서 Residual Block은 반드시 **[Weight - ReLU - Weight]** 처럼 중간에 비선형 함수가 끼어있는 2개 이상의 층으로 구성해야 한다.

- 마지막에 ReLU를 한 번 쓴다고 가정해 보자.
 - 구조: $x \rightarrow W_1 \rightarrow \text{Add } x \rightarrow \text{Final ReLU}$
 - 수식: $y = \sigma(W_1 \cdot x + x)$
 - 통계학적 해석 (결합 법칙):
여기서 x 를 묶어낼 수 있다. $y = \sigma((W_1 + I) \cdot x)$
여기서 $(W_1 + I)$ 를 그냥 새로운 행렬 W' 라고 부르자. $y = \sigma(W' \cdot x)$
이건 그냥 우리가 아는 일반적인 1층짜리 신경망(Plain Layer)이랑 수식이 100% 같다.
 - ResNet의 의도 : 기존 지식(x)은 놔두고, 부족한 부분(F)만 따로 학습하자! (분리 효과)
 - 1개 층일 때 : x 랑 F 가 선형 결합되어 버려서, 그냥 통쳐서 하나의 행렬(W')을 학습하는 꼴이 된다. 즉, 잔차 학습의 효과(분리)가 사라지는 것이다.

5.4. Network Architecture

- 설계 철학 (VGG 스타일 계승)
 - 3×3 Convolution 위주 사용.
 - 규칙 : "Feature Map 사이즈가 반으로 줄면($\frac{1}{2}$), 필터 수는 2배로 늘린다($\times 2$)."
◦ 이유 : 층마다 연산량(Time Complexity)을 일정하게 유지하기 위함.
 - Downsampling : Pooling 대신 Stride 2 Convolution 사용.
 - 효율성 : VGG-19 대비 연산량이 18%밖에 되지 않음.
- 차원(Dimension) 불일치 해결법
 - 점선(Dotted Line) 부분처럼 입력과 출력의 크기가 다를 때 처리 방법.

- 옵션 A (Zero Padding): 그냥 0을 채워 넣음. 파라미터 추가 없음.
- 옵션 B (Projection): 1×1 Conv를 써서 차원을 맞춰줌. 성능은 미세하게 더 좋음.



Example network architectures for ImageNet

5.5. 더 직관적인 예시

5.5.1. 기준 모델 (Plain Network) : VGG의 다이어트 버전

이 논문은 맨땅에서 모델을 만든 게 아니라, 당시 제일 잘나가던 **VGG 모델**을 본떠서 만들었다. 하지만 훨씬 **가볍게** 만들었다.

- **규칙 : 땅이 좁아지면 건물을 높게 올린다**

- 이미지가 층을 지날수록 크기(가로/세로)는 점점 작아진다.
- 대신 **채널(Filter, 깊이)** 수는 2배로 늘린다.
- **이유** : 이미지 크기가 반으로 줄면(1/4), 채널을 2배 늘려서 연산량을 일정하게 유지하려는 것이다.

- **특징 : 버리지 않는다 (No Pooling)**

- 보통 이미지를 줄일 때 Max Pooling이라고 해서 픽셀을 버리는 방식을 썼는데, 여기서는 그냥 보폭을 크게 성큼성큼 걷는 방식(Stride 2)을 써서 이미지를 줄였다. 이게 더 효율적이다.

5.5.2. 제안 모델 (Residual Network) : 지름길 뚫기

위에서 만든 Plain 모델에 지름길(Shortcut)만 뚫어주면 ResNet이다. 그런데 문제가 하나 있다.

문제 상황:

$F(x) + x$ 를 하려면 둘의 차원이 똑같아야 더할 수 있다(행렬이론). 그런데 층을 지나면서 채널 수가 64개 → 128개로 늘어나는 구간이 있다. 이때 어떻게 더할까?

논문에서는 두 가지 해결책을 제시한다.

옵션 A : Zero Padding

- **방법** : 64개는 그대로 두고, 나머지 부족한 64개 공간을 그냥 0(Zero)으로 채워서 128개를 만든다.
- **장점** : 계산할 게 없다. 파라미터가 0개이다.
- **성능** : 의외로 이렇게만 해도 성능이 아주 잘 나오는게 확인됐다.

옵션 B : Projection

- **방법** : 1×1 Convolution이라는 기술을 써서, 수학적으로 계산해서 64개를 128개로 변환해 준다.
- **장점** : 정보 손실 없이 더 정교하게 맞출 수 있다.
- **단점** : 계산을 해야 하니 파라미터가 조금 늘어난다.

5.6. <Tip & Question>

Q1. 이미지가 왜 자꾸 작아질까?

A. 숲을 보기 위해서이다.

- **비유** : 아주 큰 모자이크 그림이 있다고 상상해 보자.
 - **맨 앞단 (큰 이미지)** : 코앞에서 본다. 점 하나하나(Pixel)는 보이지만, 이게 '고양이'인지는 모른다. 여기서는 "**가로 세로 선이 있다**" 정도의 디테일만 본다.
 - **뒷단 (작은 이미지)** : 10미터 뒤로 물러난다. 점들은 뭉개져서 안 보이지만(해상도 감소), 대신 "**귀가 뾰족하고 수염이 있네**" 하는 전체적인 형체(Feature)가 보인다.
- **원리** : 딥러닝은 뒤로 갈수록 **불필요한 디테일(위치 정보)은 버리고**, 핵심 정보(이게 뭐다?)만 남겨야 하기 때문에 이미지를 점점 압축하는 것이다.

Q2. 크기가 반이면 1/2 아닌가? 왜 1/4로 줄었다고 할까?

A. "면적(Area)"을 생각해야 한다.

- **수학**: 가로(W)가 1/2, 세로(H)가 1/2이 되면, 전체 면적($W \times H$)은 $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$ 이 된다.
 - 즉, 처리해야 할 픽셀 수가 **4분의 1**로 확 줄어든 거다.

Q3. 그림 채널(Filter)은 왜 2배만 늘릴까?

A. 연산량(Complexity)을 일정하게 맞추기 위해서이다.

논문에서 Time Complexity를 보존한다고 했다.

합성곱 연산량은 대략 (면적) $\times$ (입력 채널) $\times$ (출력 채널)에 비례한다.

- **상황**:
 1. 면적이 $\frac{1}{4}$ 배로 줄다.
 2. 연산량을 똑같이 유지하려면 어딘가에서 **4배**를 늘려야 한다.
 3. 보통 입력 채널도 2배, 출력 채널도 2배 늘린다. ($2 \times 2 = 4$)
 4. 결과적으로 $\frac{1}{4} \times 4 = 1$ 로써 같게 된다.

Q4. Stride 2가 왜 효율적인가?

A. 계산하고 버리느냐(Pooling), 아예 건너뛰느냐(Stride)의 차이이다.

- **Max Pooling (기존 방식)**:
 1. 모든 픽셀을 꼼꼼하게 다 계산함 (Convolution).

2. 그다음 4개 중 가장 큰 값만 남기고 나머지 3개를 **버림**.

◦ **비효율** : 기껏 계산해 놓고 버린다.

- **Stride 2 (ResNet 방식):**

1. 처음부터 **한 칸씩 건너뛰면서** 계산함.

2. 계산 자체를 절반만 함.

◦ **효율** : 어차피 줄일 거면 처음부터 듬성듬성 계산하자.

Q5. 차원 맞출 때 0을 채운다는 게 무슨 뜻인가?

A. 두께를 맞추기 위해 빈 종이를 끼워 넣는 것이다.

ResNet은 $Input + Output$ 더하기를 해야 하는데, $Input$ 은 채널이 64장, $Output$ 은 128장이라고 가정하자. 짝이 맞지 않다.

- **Zero Padding (Option A):**

◦ 64장 뒤에 **아무것도 안 적힌 검은색 종이(0)** 64장을 그냥 갖다 붙인다.

◦ 그러면 128장이 되어 더할 수 있다.

◦ 무게(파라미터/계산)는 전혀 안 늘어나면서 부피(차원)만 뺄기해주므로 공짜이다.

Q6. 1×1 이면 점 하나 보는 건데, 이게 어떻게 128개로 늘려줄까?

A. 픽셀 하나하나에 대해 수행하는 선형 결합(Linear Combination)이다. (통계학적 정의)

이미지에서 1×1 공간(픽셀 하나)을 보지만, **깊이(채널) 전체**를 본다.

- **상황** : 내 손에 데이터 64개(채널)가 있다. 이걸 **128개**로 만들고 싶다?

- **통계학 비유 (회귀분석):**

◦ $y_1 = w_{1,1}x_1 + w_{1,2}x_2 + \dots + w_{1,64}x_{64}$ (첫 번째 새로운 특징 만들기)

◦ ...

◦ $y_{128} = w_{128,1}x_1 + \dots + w_{128,64}x_{64}$ (128번째 새로운 특징 만들기)

- **결과**: 1×1 Convolution은 공간(W, H)은 건드리지 않고, 이 **수식(가중치)** 개수만 조절해서 채널 개수를 마음대로($64 \rightarrow 128$) 바꿀 수 있는 도구이다.

Q7. 1×1 convolution의 진짜 의미는?

우리가 하고 싶은 건 64개의 정보(x)를 가지고 128개의 새로운 정보(y)를 만들어내는 것이다.

- **입력 (x):** 64개의 채널 (독립변수 $x_1, x_2, \dots, x_{64}$)
- **출력 (y):** 128개의 채널 (종속변수 $y_1, y_2, \dots, y_{128}$)

이때 1×1 Conv는 **64개의 입력 전부를 사용해서(가중치를 줘서) 새로운 값 하나를 만드는** 과정을 128번 반복하는 것이다.

수식으로 보기 (선형 결합)

y_1 부터 y_{128} 까지 각각 서로 다른 가중치(w)를 가지고 계산한다. 가중치는 0이 아니라, **학습을 통해 찾아낸 최적의 실수값들**이다.

$$y_1 = (w_{1,1} \cdot x_1) + (w_{1,2} \cdot x_2) + \dots + (w_{1,64} \cdot x_{64})$$

$$y_2 = (w_{2,1} \cdot x_1) + (w_{2,2} \cdot x_2) + \dots + (w_{2,64} \cdot x_{64})$$

$\vdots$

$$y_{128} = (w_{128,1} \cdot x_1) + (w_{128,2} \cdot x_2) + \dots + (w_{128,64} \cdot x_{64})$$

보다시피 w 가 0인 게 아니라, 128×64 개의 가중치(Parameter)가 딱 차게 존재한다. 즉, 단순 뺄셈이 아니라 64개의 정보를 잘 섞어서(Mix) 더 풍부한 128개의 정보로 재탄생시키는 과정이다.

Q8. Example network architectures for ImageNet 이미지 해석하는 법(중요!!!!!!)

1. 모델별 특징

• Left: VGG-19 (Reference)

- 기존의 대표적인 심층 신경망 모델.
- 3×3 합성곱 필터를 주로 사용하며, Max Pooling을 통해 이미지 크기를 줄인다.
- 마지막에 Fully Connected (FC) Layer가 많아 연산량이 매우 크다 (19.6 Billion FLOPs).

• Middle: 34-layer Plain Net

- VGG의 설계 철학을 계승하되 더 깊게(34층) 쌓은 모델.
- Pooling 대신 Stride 2 Convolution을 사용하여 다운 샘플링을 수행한다.

- 마지막의 무거운 FC Layer를 제거하여 VGG-19보다 깊지만 연산량은 오히려 18% 수준(3.6 Billion FLOPs)으로 가볍다.

- **Right: 34-layer Residual Net (ResNet)**

- Plain Net에 Shortcut Connection(화살표)을 추가한 구조.
- 파라미터 수와 연산량은 Plain Net과 완전히 동일하지만, 최적화가 훨씬 용이하다.

2. 화살표(Shortcut)의 의미

오른쪽 ResNet 그림에 있는 곡선 화살표는 두 가지 형태로 구분된다.

- **실선 (Solid Line Shortcuts):**

- 입력과 출력의 차원(크기 및 채널 수)이 동일한 구간.
- 별도의 처리 없이 $F(x) + x$ 연산을 수행한다. (Identity Mapping)

- **점선 (Dotted Line Shortcuts):**

- **차원이 변하는 구간** (예: 64 필터 → 128 필터).
- 이미지 크기는 반으로 줄어든다($/2$), 필터 수는 2배로 늘어난다.
- 이때 차원을 맞춰주기 위해 1×1 **Convolution (Projection)** 또는 **Zero Padding**을 적용한 후 더해준다.

3. 설계 규칙 (VGG Philosophy)

모든 모델은 다음의 규칙을 따른다.

- **Output Size가 같으면:** 동일한 개수의 필터를 사용한다.
- **Output Size가 반으로 줄면:** 필터(채널) 수를 2배로 늘려 레이어당 연산 복잡도(Time Complexity)를 일정하게 보존한다.

5.7. Implementation

바로 위에서 보았듯이 이러한 다양한 가중치 값을 어떻게 학습할 수 있을까?

- **데이터 전처리 (Data Augmentation) - "데이터 뺑튀기"**

이미지를 그냥 넣는 게 아니라, 모델이 다양한 상황을 배우도록 이미지를 한다.

- **크기 조절** : 이미지를 256x256이나 480x480 등으로 다양하게 resizing한다.
- **자르기 (Crop)** : 그 중에서 **224x224** 크기로 무작위로 잘라낸다. (가장 표준적인 입

력 크기)

- **뒤집기 (Flip)** : 이미지를 좌우 반전시키기도 한다.
- **색상 변형 (Color Jittering)** : 밝기, 색상, 채도 등을 살짝씩 바꾼다.

- **정규화 (Normalization)**

- **Batch Normalization (BN)** : 합성곱(Conv) 바로 뒤, 활성화 함수(ReLU) 바로 앞에 BN을 사용했다.
- **특징** : BN이 학습을 안정화해주기 때문에, 기존에 과적합 방지용으로 쓰던 **Dropout** 을 쓰지 않았다. (이게 중요한 포인트이다)

- **최적화 설정 (Hyperparameters)**

통계학과에서 회귀분석 돌릴 때 파라미터 세팅하는 거랑 비슷하다.

- **초기화 (Initialization)** : 가중치(W)를 무작위로 두는 게 아니라 **He Initialization** 이라는 방법을 써서 예쁘게 초기화하고 시작한다.
- **Optimizer** : SGD (확률적 경사 하강법)
- **Batch Size** : 256 (한 번에 문제 256개씩 풀기)
- **Learning Rate (학습률)** : 0.1에서 시작. 에러가 더 이상 안 줄어들면 1/10로 줄임.
- **Momentum** : 0.9 (관성 적용)
- **Weight Decay** : 0.0001

즉, ResNet은 꼼수(Dropout 등)를 다 빼고, BN(Batch Norm)과 정석적인 SGD만으로도 152층의 가중치들을 완벽하게 학습시켰다.

6. 논문 심층 분석 4 - Experiments

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Architectures for ImageNet

6.1. Table 1 상세 분석 : 아키텍처 구성 원리

ResNet의 아키텍처를 보여주는 Table 1은 모델의 깊이(18, 34, 50 등)에 따라 **블록의 종류**와 **반복 횟수**가 어떻게 달라지는지를 명시한 설계도이다. 이를 해석하는 방법은 다음과 같다.

6.1.1 표기법 해석 (ResNet-34를 중심으로)

표의 conv3_x 행에 기재된 다음 수식을 분석한다.

$$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$$

- **대괄호 [] 내부 (블록 구성)** : Residual Block 1개를 구성하는 레이어의 정보이다.
 - 3×3 : 필터(커널)의 크기.
 - 128 : 채널(필터)의 개수.
 - 두 줄로 표기된 이유 : 이 블록 하나에 3×3 합성곱 층이 **2개** 포함되어 있음을 의미한다. (Basic Block)
- **우측의 x4 (반복 횟수)** : 해당 블록을 **4번** 반복하여 쌓는다는 의미이다.
- **총수 계산** : (블록 당 2개 층) $\times$ (4번 반복) = **총 8개 층**이 해당 구역에 생성된다.

6.1.2. 구조적 차이점 (ResNet-50 예시)

ResNet-50 이상의 모델에서는 conv3_x의 표기가 다음과 같이 변경된다.

$$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$$

- **대괄호 [] 내부:** 층이 3줄로 늘어났다. 이는 **Bottleneck Block**을 사용함을 의미하며, 블록 1개당 **3개의 층**($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$)이 포함된다.
- **반복 횟수 :** 동일하게 **4번** 반복한다.
- **층수 계산 :** (블록 당 3개 층) $\times$ (4번 반복) = **총 12개 층**이 생성된다.
- **특이점 :** 블록 내부 구조의 차이(2줄 vs 3줄)로 인해 같은 반복 횟수라도 전체 깊이 (Depth)가 달라지게 된다.

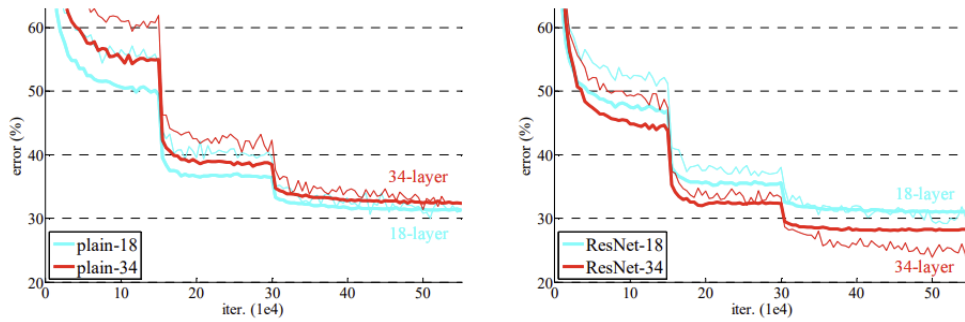
6.1.3. 전체 층수(Depth) 계산

각 단계(Stage)별 반복 횟수 리스트를 통해 모델의 최종 이름(18, 50 등)이 도출되는 과정이다.

- **ResNet-18 (Basic Block: 블록당 2개 층)**
 - 반복 횟수: **[2, 2, 2, 2]**
 - 계산: $(2 + 2 + 2 + 2) \times 2 = 16$ 개 층.
 - 최종: $16 + 2$ (시작 Conv1 + 끝 FC) = **18-layers**.
- **ResNet-50 (Bottleneck Block: 블록당 3개 층)**
 - 반복 횟수: **[3, 4, 6, 3]**
 - 계산: $(3 + 4 + 6 + 3) \times 3 = 48$ 개 층.
 - 최종: $48 + 2$ (시작 Conv1 + 끝 FC) = **50-layers**.

6.2. 실험 결과

본 논문에서는 ImageNet 데이터셋을 사용하여 Plain Network(기존 방식)와 ResNet(제안 방식)의 성능을 비교 분석하고, 퇴보 문제(Degradation Problem)가 해결되었음을 입증하였다.



Training on ImageNet

6.2.1. 퇴보 문제의 확인 (Plain Networks)

- **실험 개요** : 18층과 34층의 일반 네트워크(Plain Net)를 비교함.
- **결과 (Figure 4 좌측)** : 34층 모델이 18층 모델보다 학습 에러(Training Error)와 검증 에러(Validation Error)가 모두 높게 나타남.
- **분석**:
 - 학습 에러가 높다는 것은 과적합(Overfitting) 문제가 아님을 의미한다.
 - Batch Normalization(BN)을 적용했으므로 기울기 소실(Vanishing Gradient) 문제도 아니다.
 - 결론적으로, 단순히 층을 깊게 쌓는 구조는 **최적화(Optimization) 난이도가 기하급수적으로 상승함**을 시사한다.

6.2.2. 잔차 학습의 효과 (Residual Networks)

- **실험 개요** : 동일한 깊이(18층, 34층)의 ResNet 모델을 비교함.
- **결과 (Figure 4 우측)**:
 - **ResNet-34가 ResNet-18보다 더 낮은 에러율을 기록함.** (Plain Network와 정반대 결과)
 - Plain-34와 비교했을 때, ResNet-34는 Top-1 에러를 3.5% 포인트 감소시킴.
- **의의**:
 - Shortcut Connection을 통해 최적화 난이도가 낮아져, 깊은 모델도 쉽게 학습할 수 있음을 증명함.
 - 층이 깊어질수록 성능이 향상되는 깊이의 이득(Depth Gain)을 실제로 얻을 수 있게 됨.

6.2.3. 최신 모델과의 비교 (SOTA 달성)

- **ResNet-152:** 병목 구조(Bottleneck Architecture)를 사용하여 깊이를 152층까지 늘린 모델.
- **성능:** ImageNet 테스트 셋에서 **3.57%**의 Top-5 에러율을 달성함.
- **결과:** VGG, GoogLeNet 등 기존 모델을 큰 격차로 따돌리고 **ILSVRC 2015 대회에서 1위**를 차지함. 인간의 분류 에러율(약 5%)보다 낮은 수준이다.

6.3. 중요 개념 소개 - Identity vs. Projection & Bottleneck

6.3.1. 추가 실험 : Shortcut 옵션 비교

차원(Dimension)이 변경될 때 Shortcut을 연결하는 최적의 방법을 찾기 위해 세 가지 옵션을 비교 실험하였다.

- **세 가지 옵션 정의**
 - **Option A (Zero-padding)** : 차원이 늘어난 부분을 0으로 채움. 파라미터가 추가되지 않음.
 - **Option B (Projection - Partial)** : 차원이 늘어날 때만 1×1 Convolution을 사용하여 차원을 맞춤. (ResNet의 표준 방식)
 - **Option C (Projection - All)** : 모든 Shortcut 구간에 1×1 Convolution을 적용함.
- **실험 결과 (Table 3 분석)**
 - **성능 순서:** $C > B > A$ (하지만 차이는 미미함).
 - **분석:**
 - B가 A보다 우수한 이유 : Zero-padding된 부분은 잔차(Residual) 정보를 갖지 못하기 때문에 학습 효율이 떨어짐.
 - C가 B보다 우수한 이유 : 단순히 파라미터 수가 늘어났기 때문이며, 잔차 학습의 본질적인 효과 차이는 아님.
 - **결론 :** 파라미터와 메모리 효율성을 고려하여, 성능 차이가 크지 않은 C 대신 **Option B**를 표준으로 채택한다. (단, 아주 가벼운 모델이 필요할 땐 A도 유효함)

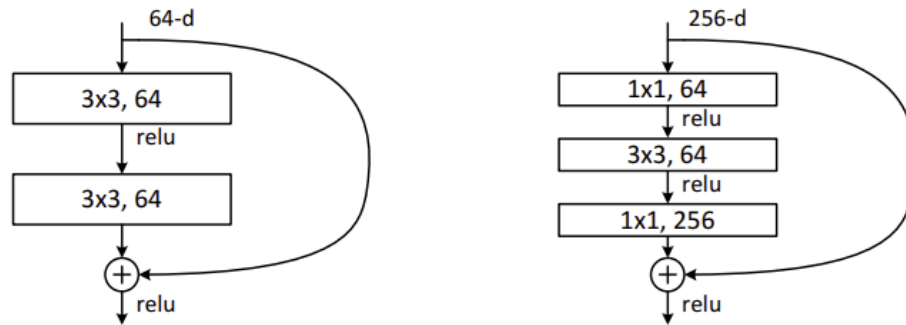


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Basic block VS Bottleneck Block

6.3.2. 심층 모델을 위한 Bottleneck 구조

ResNet-50 이상의 깊은 모델에서는 연산량(FLOPs)을 제어하기 위해 블록의 구조를 변경하였다.

- 구조 비교 (Figure 5)
 - Left (Basic Block):
 - 사용 모델: ResNet-18, 34
 - 구조: $3 \times 3 \rightarrow 3 \times 3$ (2단 구조)
 - Right (Bottleneck Block):
 - 사용 모델: ResNet-50, 101, 152
 - 구조: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ (3단 구조)
- 병목(Bottleneck) 설계의 원리
 - 1×1 Conv (축소) : 입력 채널(256-d)을 64-d로 줄여서 3×3 연산의 부담을 줄인다.
 - 3×3 Conv : 줄어들거나 압축된 차원에서 특징을 추출한다.
 - 1×1 Conv (확장) : 다시 채널을 256-d로 복원하여 다음 블록과 차원을 맞춘다.
 - 효과 : 이 구조 덕분에 152층까지 쌓아도 VGG-19보다 연산량이 적은 효율적인 모델이 가능해졌다.

6.3.3. 병목 구조의 효율성 원리 (Why Bottleneck?)

- **도입 배경:** 모델이 깊어질수록(50층 이상) 학습 시간이 지나치게 길어지는 문제를 해결하기 위함.

- **작동 메커니즘 (1×1 Conv의 활용):**

- 1. **Reduce (1×1):** 차원을 먼저 줄여서 연산량을 감소시킴. (예: $256 \rightarrow 64$)

- 2. **Process (3×3):** 줄어든 차원에서 핵심 연산 수행. (병목 구간)

- 3. **Restore (1×1):** 다시 차원을 원래대로 복구함. (예: $64 \rightarrow 256$)

- **결과 :** 층수는 3개로 늘어났지만, 전체 연산량(Time Complexity)은 기존 2개 층 구조(Basic Block)와 유사하게 유지됨.

6.3.4. Identity Shortcut의 필수성

- **중요한 발견 :** 병목 구조에서는 파라미터가 없는 Identity Shortcut(실선)을 사용하는 것이 매우 중요하다.

- **이유:**

- 병목 구조는 양 끝단(입출력)의 차원이 매우 높다(예: 256, 1024 등).

- 만약 이곳을 Projection Shortcut(1×1 Conv 등)으로 연결하면, 연결하는 데만 막대한 연산량이 소모된다.

- **결론 :** Projection을 남발하면 모델 크기와 연산량이 **2배**로 폭증하므로, 병목 구조에서는 Identity Shortcut이 효율성의 핵심이다.

6.3.5. <Tip> 더 쉬운 비유로 이해해보자!

6.3.5.1. 선택(Selection) vs 압축(Compression)

- **상황 :** 256개의 채널이 있음.

- **압축 (새로운 채널 1 만들기) :** 256개 채널 전부(X_{all})에 각각 다른 가중치(w)를 곱해서 하나로 더한다.

$$Y_1 = (w_1 \cdot X_1) + (w_2 \cdot X_2) + \dots + (w_{256} \cdot X_{256})$$

- **결과 :** 이렇게 만들어진 Y_1 안에는 X_1 부터 X_{256} 까지의 정보가 아주 조금씩이라도 **다** 녹아들어 있다.

6.3.5.2. 비싼 전문가에게 요약본만 보여주기

상황을 가정해 보자. 256페이지짜리 보고서를 가지고 있고, 이것 분석해야 한다.

- **기존 방식 (Basic Block):**

- **상황:** 이 256페이지(256 Channels)를 전부 꼼꼼하게 읽고 분석한다(3×3 Conv).
- **문제 :** 시간이 너무 오래 걸리고 비용이 비싸다.

- **병목 방식 (Bottleneck):**

1. **압축 (1×1) :** 일단 조수가 256페이지를 핵심만 추려서 **64페이지로 요약한다.** (차원 축소)
2. **연산 (3×3):** 이제 전문가가 이 **64페이지**만 꼼꼼하게 분석한다. (핵심 연산 수행 -> 여기서 비용 대폭 절감)
3. **복원 (1×1) :** 분석된 내용을 바탕으로 다시 256페이지 분량의 상세 보고서로 올려준다. (차원 복구)

6.3.5.3. 숫자로 보는 가벼워짐의 증거 (수학적 증명)

실제 연산량(파라미터 수)을 계산해서 비교해 보자.

입력이 **256 채널**이고, 출력도 **256 채널**을 유지해야 하는 상황이라고 가정하자.

A. 기존 방식 (그냥 3×3 으로 밀어붙이기)

- 3×3 커널을 사용하여 256개를 256개로 보냄.
- **계산식:** $3 \times 3 \times 256 \times 256$
- **파라미터 수 :** 약 **600,000개** (589,824)

B. 병목 방식 (1×1 로 줄였다가 키우기)

이걸 3단계로 쪼개서 계산해 본다. (중간에 64채널로 줄임)

1. **압축 (1×1):** $256 \rightarrow 64$
 - $1 \times 1 \times 256 \times 64 \approx$ **16,000개**
 2. **연산 (3×3):** $64 \rightarrow 64$ (여기가 핵심! 입력이 작아서 연산량이 확 준다)
 - $3 \times 3 \times 64 \times 64 \approx$ **36,000개**
 3. **복원 (1×1):** $64 \rightarrow 256$
 - $1 \times 1 \times 64 \times 256 \approx$ **16,000개**
- **총합:** $16,000 + 36,000 + 16,000 =$ **약 70,000개**

3. 결론: 압축의 마법

- **Before:** 600,000개 연산
- **After:** 70,000개 연산
- **결과:** 성능은 비슷한데 연산량은 **거의 9배(1/9) 가까이 줄어들었다.**

6.4. 최종 성능 및 SOTA 비교 (Comparisons with SOTA)

Bottleneck 구조를 적용한 심층 모델(50, 101, 152-layer)의 성능을 평가하고, 기존 최고 성능 모델(State-of-the-art)들과 비교하였다.

1. 깊이에 따른 성능 향상 (No Degradation)

- **구조:** 50, 101, 152층 모델은 앞서 설계한 3-layer Bottleneck Block을 사용하여 구축하였다.
- **결과:**
 - 층이 깊어질수록(34 → 50 → 101 → 152) 에러율이 지속적으로 감소하였다.
 - 이는 기존 Plain Network에서 발생하던 **퇴보(Degradation) 문제가 완벽하게 해결**되었음을 의미하며, 깊이(Depth)가 주는 이득을 온전히 누릴 수 있게 되었다.

2. 놀라운 효율성 (Efficiency)

- **비교 :** ResNet-152는 VGG-19보다 깊이가 약 8배나 깊다. (152층 vs 19층)
- **연산량(Complexity):**
 - **VGG-19:** 19.6 Billion FLOPs
 - **ResNet-152:** 11.3 Billion FLOPs
- **결론 :** Bottleneck 구조 덕분에 152층 모델이 19층 모델보다 **오히려 연산량이 적다.** 이는 모델의 효율성 측면에서 비약적인 발전을 이뤘음을 시사한다.

3. ILSVRC 2015 결과 (SOTA 달성)

- **단일 모델(Single Model) :** ResNet-152 단일 모델만으로도 Top-5 에러율 4.49%를 기록하며, 이전의 모든 앙상블(Ensemble) 기록을 경신했다.
- **앙상블(Ensemble) :** 서로 다른 깊이의 ResNet 모델 6개를 앙상블한 결과, Top-5 에러율 3.57%를 기록하였다.
- **의의 :** 이 기록으로 ILSVRC 2015 대회에서 1위를 차지하였으며, 인간의 분류 오차율(약 5%)을 뛰어넘는 성능을 입증하였다.

6.5. 추가 분석 : CIFAR-10 및 층 반응(Layer Response)

6.5.1. 층 반응 분석 (Analysis of Layer Responses)

- **실험 목적** : 잔차 학습($F(x)$)이 0에 가깝게 학습될 것이라라는 초기 가설이 실제로 맞는 지 데이터로 검증함.
- **지표** : 각 3×3 합성곱 층 출력값의 표준편차(Standard Deviation, std)를 측정.
- **결과**:
 - **ResNet** : 층이 깊어질수록 출력값의 표준편차가 **일반 네트워크(Plain)보다 훨씬 작음**을 확인하였다.
 - **해석** : ResNet의 $F(x)$ 는 실제로 0에 가까운 작은 값(Minor modification)만을 학습하고 있음을 의미한다. 즉, 입력(x)을 거의 그대로 보존하면서($+x$), 필요한 정보만 살짝 수정($+F(x)$)한다는 가설이 통계적으로 입증되었다.

6.5.2. 1000층 이상의 탐색 (Exploring Over 1000 layers)

- **실험 개요** : 1202층짜리 극도로 깊은 ResNet 모델을 구축하여 학습을 시도함.
- **결과 (Optimization)**:
 - 학습 에러(Training Error)가 0.1% 미만으로 수렴함.
 - 이는 1000층이 넘어도 최적화(Optimization) 자체에는 전혀 문제가 없음을 보여준다. (기울기 소실/폭주 없음)
- **결과 (Generalization)**:
 - 하지만 테스트 에러(Test Error)는 110층 모델보다 오히려 높게 나타남(7.93%).
- **원인 분석 (Overfitting)**:
 - 이는 모델의 깊이로 인한 퇴보(Degradation) 문제가 아니라, CIFAR-10 데이터셋의 크기에 비해 모델이 너무 커서 발생한 **과적합(Overfitting)** 현상이다.
 - 즉, 데이터만 충분하다면 ResNet은 1000층 이상도 문제없이 학습할 수 있는 잠재력이 있다.

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean±std)” as in [43].

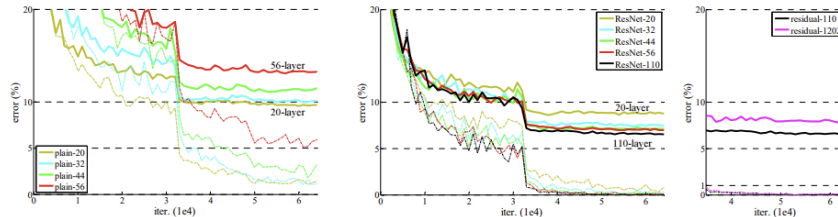


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

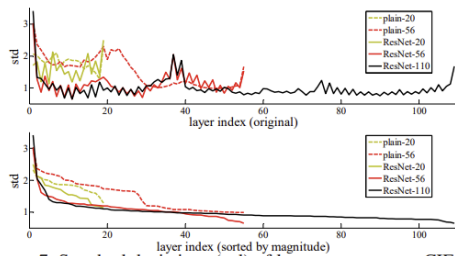


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

7. 결론 및 제언

본 논문은 딥러닝 모델의 깊이(Depth)가 깊어질수록 오히려 성능이 떨어지는 퇴보 문제(Degradation Problem)를 규명하고, 이를 잔차 학습(Residual Learning)이라는 혁신적인 프레임워크로 해결하였다.

7.1. 핵심 요약 (Summary)

- **문제 정의** : 네트워크의 층이 깊어지면 과적합(Overfitting)이 아님에도 불구하고 학습 에러(Training Error)가 증가하는 최적화(Optimization) 난제가 발생함을 확인하였다.
- **해결책** : 입력(x)을 출력에 그대로 더해주는 **Shortcut Connection**을 도입하여, 네트워크가 복잡한 함수($H(x)$) 전체가 아닌 나머지(Residual, $F(x)$)만을 학습하도록 유도하였다.
- **결과** : 152층에 달하는 초심층 모델(ResNet)을 성공적으로 학습시켰으며, ImageNet, COCO 등 주요 대회에서 압도적인 성능으로 SOTA를 달성하였다.

7.2. 논문의 주요 기여점 (Main Contributions)

- **최적화 관점의 전환**:
 - 기존에는 모델 성능 저하의 원인을 파라미터 과다로 인한 과적합(Overfitting)에서 찾으려 했으나, 본 논문은 이를 최적화의 어려움(Optimization Difficulty)으로 재정의하였다.
 - Shortcut Connection은 기울기(Gradient)가 소실되지 않고 네트워크 하단까지 직접 전파되는 '**고속도로(Gradient Highway)**' 역할을 수행하여 이 문제를 해결하였다.
- **구조적 효율성 (Efficiency)**:
 - Bottleneck 구조(1×1 Conv)를 도입하여, 채널을 압축-연산-복원하는 방식으로 연산량(FLOPs)을 제어하였다.
 - 이로 인해 VGG-19보다 8배 깊은 152층 모델임에도 불구하고, 실제 연산량은 오히려 더 적은 고효율 아키텍처를 완성하였다.

7.3. 통계학적/분석적 고찰 (Discussion)

- **가설 검증의 완결성**:

◦ 단순히 "성능이 올랐다"는 결과 제시에 그치지 않고, 층별 출력의 **표준편차(std)** 분석을 통해 "잔차 함수($F(x)$)의 반응이 0에 가깝게 작아진다"는 초기 가설을 통계적으로 입증한 점이 돋보인다.

• **앙상블(Ensemble) 효과에 대한 해석:**

◦ ResNet은 하나의 거대한 깊은 네트워크처럼 보이지만, 실제로는 수많은 얇은 네트워크들이 Shortcut을 통해 얹혀있는 경로들의 앙상블(Ensemble of Paths)로 해석될 여지가 있다.

◦ 이는 특정 층을 제거해도 전체 성능이 급격히 무너지지 않는다는 점에서, 정보가 흐를 수 있는 우회로가 다수 존재함을 시사한다.

8. 부록 : 파이토치(PyTorch) 구현 예시

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=
1):
        super(ResidualBlock, self).__init__()

        # 1. F(x)의 첫 번째 층 (Weight Layer 1)
        # 3x3 합성곱, 차원 유지를 위해 padding=1 사용
        self.conv1 = nn.Conv2d(in_channels, out_channels, k
ernel_size=3,
                                stride=stride, padding=1, bi
as=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        # 2. F(x)의 두 번째 층 (Weight Layer 2)
        self.conv2 = nn.Conv2d(out_channels, out_channels,
kernel_size=3,
                                stride=1, padding=1, bias=Fa
lse)
        self.bn2 = nn.BatchNorm2d(out_channels)
```

```

# 3. Shortcut Connection (차원 맞추기용)
# 만약 입력(x)과 출력(F(x))의 차원(채널 수 or 크기)이 다
r면,
# 1x1 Conv로 맞춰주는 작업(Projection, Option B)이 필
요하다.
self.shortcut = nn.Sequential()
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel
_size=1,
                    stride=stride, bias=False),
        nn.BatchNorm2d(out_channels))

def forward(self, x):
    identity = x # [핵심] 입력을 변형시키지 않고 따로 저장해
둔 (Identity)

    # F(x) 계산 과정 (Weight Layer들을 통과)
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)

    # [화룡점정] Residual Addition
    # F(x) 결과에 아까 저장해둔 x(identity)를 그대로 더한다.
    out += self.shortcut(identity)

    out = self.relu(out) # 마지막에 ReLU 통과

    return out

if __name__ == "__main__":

```

```

# 시나리오
# - 입력 채널: 64
# - 목표 채널: 128 (2배 확장)
# - Stride: 2 (이미지 크기 절반 축소)
block = ResidualBlock(in_channels=64, out_channels=128,
stride=2)

# 가상의 이미지 데이터 생성 (Random Tensor)
# 모양: [Batch Size=1, Channel=64, Height=32, Width=32]
x = torch.randn(1, 64, 32, 32)

# 블록 통과 (Forward Pass)
# 내부적으로 1x1 Conv Shortcut이 작동하여 차원을 맞춘 뒤 더해
준다.
output = block(x)

# 결과 확인
# 기대 결과: [1, 128, 16, 16]
print(f"Step 3: 출력 데이터 모양 확인: {output.shape}")

# 지금까지 만든 Feature Map은 이미지를 컴퓨터가 이해할 수 있는
# 숫자 형태(Vector)로 번역해 놓은 것이다.

```